

Implementation and Verification of a CAN/CAN FD Protocol Controller in VHDL

Mads Richardt (s224948)

May 18, 2026

Abstract

This thesis describes the design, implementation, and verification of a CAN/CAN FD protocol controller in VHDL, targeting high-reliability engine controller applications at Everllence. The controller complies with ISO 11898-1 and supports the CB, CE, FB, and FE frame formats with dual bit rate switching and Transmitter Delay Compensation (TDC) for the FD data phase. The design is structured around the ISO 11898-1 layered reference model, with the MAC, PCS, and FCE sub-layers implemented as independently testable modules and the LLC sub-layer specified but deferred. Of 38 derived requirements, 28 are verified against passing testbenches or code inspection. Of the remaining 10, seven are deferred pending `can_llc` implementation, two are non-blocking P2 items, and one ([REQ-022](#)) has partial simulation coverage with dedicated error-injection testing identified as future work. The design was synthesized on a Cyclone 10 LP FPGA target, using 4,608 logic elements (30% of device) with a worst-case f_{max} of approximately 127 MHz.

Approval

This thesis was prepared at the Department of Applied Mathematics and Computer Science (DTU Compute), Technical University of Denmark, in partial fulfilment of the requirements for the degree of Bachelor of Science in Engineering. The work was carried out in collaboration with Everllence, where the controller is intended for use in high-reliability engine controller applications.

It is assumed that the reader has a working knowledge of digital logic design and binary communication protocols. Familiarity with VHDL or a similar hardware description language is an advantage but is not strictly required to follow the architectural and verification discussions.

Mads Richardt (s224948)

Kgs. Lyngby, May 2026

Acknowledgements

Edward Alexandru Todorica, Associate Professor, DTU Compute. Thank you for supervision throughout the project and for valuable guidance on verification methodology and VHDL design practice.

Fredrik Kristensen, Everllence. Thank you for framing the industrial requirements, providing access to the existing CAN controller as a reference, and for feedback on the design and implementation.

Contents

Approval	2
Acknowledgements	3
Abbreviations	7
List of Figures	9
1 Introduction	11
1.1 Motivation	11
1.2 Existing CAN Controller	11
1.2.1 Limitations	11
1.2.2 Decision to Redesign	12
1.3 Existing CAN FD IP Cores	12
1.3.1 Open-Source Implementations	12
1.3.2 Commercial Implementations	12
1.3.3 Rationale for In-House Development	13
1.4 Problem Statement	14
1.5 Objectives	14
2 Background	14
2.1 CAN as a Communication Bus	14
2.2 Tools and Language	16
3 Requirements	16
3.1 VHDL Code Standard and Design Constraints	16
3.1.1 Project-specific Infrastructure Requirements	17
3.1.2 File Structure and Naming	17
3.1.3 RTL Coding Style and Verification	17
3.2 From Specification to Structured Requirements	17
3.3 AI-Assisted Extraction: Utility and Limitations	19
4 CAN and CAN FD Protocol Overview	20
4.1 Layered Reference Model	20
4.2 Frame Types and Formats	21
4.3 Bit Timing and Flexible Data Rate	21
4.4 Bit Stuffing	24
4.5 Cyclic Redundancy Check	24
4.6 Error Detection and Fault Confinement	25
5 Verification Plan	25
5.1 Layer	26
5.2 Side	26
5.3 Format Applicability	26
5.4 Observability	26
5.5 Priority	27
5.6 Requirement Distribution	27

5.7	Verification Plan Data Structure	27
5.7.1	Verification Method	28
5.7.2	Traceability: Label and File	28
5.7.3	Status	28
6	Design and Architecture	29
6.1	Ramifications of the Requirements Model on Initial Design Strategy	29
6.2	Architectural Design Decisions	30
6.2.1	Adopting the ISO 11898-1 Sub-layer Model	30
6.2.2	Combined vs. Separated TX/RX FSMs	30
6.2.3	Per-Field vs. Per-Phase FSM Granularity	31
6.2.4	Internal LLC Frame Format	32
6.3	System Overview	32
6.4	LLC Frame Format	32
6.5	LLC Sub-layer	33
6.6	MAC Sub-layer	33
6.6.1	can_mac_fsm	34
6.6.2	can_mac_ser	34
6.6.3	can_mac_bs	34
6.6.4	can_mac_crc	34
6.7	FCE Sub-layer	34
6.8	PCS Sub-layer	35
7	Implementation	35
7.1	Interface Conventions	35
7.2	can_mac_fsm	36
7.2.1	FSM Structure and Mode Flag	36
7.2.2	TX Mode: Frame Transmission	36
7.2.3	RX Mode: Frame Reception	39
7.2.4	Error-Frame States	39
7.3	can_mac_ser	40
7.4	can_mac_bs	41
7.5	can_mac_crc	42
7.6	can_fce	42
7.7	can_pcs	43
7.7.1	Resynchronization	44
7.7.2	Dual Bit Rate Switching	44
7.7.3	Transmitter Delay Compensation	46
8	Verification and Results	46
8.1	Testbench Results Summary	49
9	Synthesis	49
9.1	Resource Utilization	49
9.2	Timing Results	50
10	Discussion	50

10.1 Objectives Assessment	51
10.2 Future Work	52
11 Conclusion	53
12 References	53
A Verification Plan	54

Abbreviations

Table 1: Abbreviations used in this report.

Abbreviation	Meaning
ACK	Acknowledgment
AD	ACK Delimiter
AEF	Active Error Flag
AI	Artificial Intelligence
BRS	Bit Rate Switch
CAN	Controller Area Network
CANH	CAN High bus wire
CANL	CAN Low bus wire
CB	Classic Base (frame format)
CC	CAN Classic
CD	CRC Delimiter
CE	Classic Extended (frame format)
CRC	Cyclic Redundancy Check
DLC	Data Length Code
DMA	Direct Memory Access
DUT	Device Under Test
ED	Error Delimiter
EOF	End of Frame
ESI	Error State Indicator
FB	FD Base (frame format)
FCE	Fault Confinement Entity
FD	Flexible Data Rate
FDF	FD Frame bit
FE	FD Extended (frame format)
FPGA	Field-Programmable Gate Array
FSB	Fixed Stuff Bit
FSM	Finite State Machine
FTYP	Frame Type
HDL	Hardware Description Language
IDB	Identifier Base field (11-bit base ID)
IDE	Identifier Extension bit
IEXT	Identifier Extension field (18-bit extended ID)
IFS	Inter Frame Space
INT	Intermission
IP	Intellectual Property
ISO	International Organization for Standardization
LLC	Logical Link Control
LLM	Large Language Model
MAC	Medium Access Control
MIT	Massachusetts Institute of Technology (license)

Abbreviation	Meaning
MSB	Most Significant Bit
OD	Overload Delimiter
OF	Overload Flag
OSVVM	Open Source VHDL Verification Methodology
PCS	Physical Coding Sublayer
PE	Phase Error
PEF	Passive Error Flag
PS	Propagation Segment (PROP_SEG)
PS1	Phase Segment 1 (PHASE_SEG1)
PS2	Phase Segment 2 (PHASE_SEG2)
RRS	Reserved Remote Request Substitution bit (FD frames)
RTL	Register Transfer Level
RTR	Remote Transmission Request
RX	Receiver / Receive
SB	Stuff Bit
SBC	Stuff Bit Count
SJW	Synchronization Jump Width
SOF	Start of Frame
SP	Sample Point
SRR	Substitute Remote Request
SS	Sync Segment (SYNC_SEG)
SSP	Secondary Sample Point
ST	Suspend Transmission
TDC	Transmitter Delay Compensation
TEC/REC	Transmit Error Counter / Receive Error Counter
TQ	Time Quantum
TX	Transmitter / Transmit
VHDL	VHSIC Hardware Description Language

List of Figures

1	CAN bus with four nodes on a shared differential two-wire bus.	15
2	Pipeline generating the <code>verification_plan.toml</code> artifact from the ISO 11898-1 standard.	18
3	ISO 11898-1 CAN node reference model showing the LLC, MAC, PCS sub-layers and cross-cutting FCE.	20
4	Frame formats for the four in-scope frame types (CB, CE, FB, FE) and the error and overload flags. Field widths are annotated per ISO 11898-1.	22
5	CAN bit time structure showing the four segments (SS, PS, PS1, PS2), the sample point, and propagation delays on a two-node bus.	23
6	Resynchronization over two successive sync edges. PE is the phase error relative to SYNC_SEG. SJW is the Synchronization Jump Width.	23
7	Transmitter Delay Compensation (TDC). The loop delay t_{loop} is measured on the first data-phase bit and used to position the SSP at $t_{\text{SSP}} = t_{\text{TDC_offset}} + t_{\text{measured}}$	24
8	Dynamic and fixed bit-stuffing examples showing stuff bit placement for both encoding modes.	24
9	Internal LLC frame format at the <code>can_mac_ser</code> input, with identifier byte mapping for base and extended IDs.	32
10	Implementation module decomposition showing the five entities and their inter-module connections.	33
11	LLC frame format (71 bytes) at the host-LLC interface, with identifier byte mapping for base and extended IDs.	33
12	<code>can_mac</code> architecture with signal-level connections between the four internal entities and external interfaces.	37
13	<code>can_mac_fsm</code> (19 states) controlling TX and RX for all in-scope frame formats. Arbitration loss clears <code>is_transmitter</code> in place without a state transition.	38
14	<code>can_mac_ser</code> FSM (four states) serializing the internal LLC frame to the MAC bit stream. Unused padding bits in the 32-bit ID field are skipped silently.	40
15	<code>can_mac_bs</code> operating in dynamic and fixed stuffing modes. A pending dynamic stuff bit at the rising edge of <code>fsb_en</code> is promoted to the initial FSB rather than suppressed.	41
16	<code>can_mac_crc</code> dataflow with three parallel CRC engines. The output mux is combinatorial to avoid one cycle of read latency at the FSM.	43
17	<code>can_fce</code> FSM governing the error active, error passive, and bus off node states per ISO 11898-1 sec. 8.1.4.4.	43
18	<code>can_pcs</code> bit-time FSM with concurrent resynchronization and TDC pipelines per ISO 11898-1 sec. 7.2-7.4.	45
19	Two-node simulation of a complete FD frame, showing the full FSM field sequence on both transmitter and receiver from <code>s_arbitration</code> through bus re-integration.	47
20	Error escalation waveform showing TEC incrementing across successive ACK errors, <code>error_active</code> deasserting at the 127 threshold, and the node transitioning to error passive (REQ-030, REQ-031).	48
21	Dynamic bit stuffing in <code>can_mac_bs_tb</code> , covering both a dominant-run and a recessive-run case.	48
22	Fixed bit stuffing mode transition in <code>can_mac_bs_tb</code> . A pending dynamic stuff bit at the <code>fsb_en</code> rising edge is promoted to the initial FSB rather than suppressed, preventing TX/RX SBC divergence.	48

23	Dual bit rate switching and TDC in <code>can_pcs_tb</code> , showing the BRS transition and SSP positioning.	48
24	Arbitration loss in <code>can_mac_pcs_fce_tb</code> . The losing node clears <code>is_transmitter</code> in <code>s_arbitration</code> and continues as receiver without a state transition.	48
25	Active error frame sequence in <code>can_mac_pcs_fce_tb</code> following a forced bit error in the data field.	48
26	Bus-off recovery in <code>can_fce_tb</code> , showing the 128 idle-condition pulses required before TEC/REC reset and return to error active.	48

1 Introduction

Industrial control systems for large marine engines demand communication protocols that combine fault tolerance, multi-master arbitration, and multi-decade service reliability. The Controller Area Network has served that role at Everllence for the current generation of engine controllers, but as control system data requirements grow the bandwidth and payload limits of CAN Classic have become a bottleneck.

1.1 Motivation

Everllence (formerly MAN Energy Solutions) is a provider of propulsion and decarbonization solutions for the marine and energy industries, designing large two-stroke and four-stroke combustion engines for ship propulsion and power generation. This thesis was written within the engine controller division, where FPGA-based control hardware is developed and maintained for integration into Everllence's engine platforms.

The Controller Area Network (CAN) has been the workhorse of industrial control communication for decades, with fault-confinement properties that suit the reliability requirements of engine control applications. Everllence's existing CAN infrastructure is built around a CAN Classic protocol controller deployed on an IO-extender board forming part of the Triton motor controller system. As control systems grow more data-intensive, the eight-byte payload and 1 Mbit/s ceiling of CAN Classic has become a practical constraint. CAN FD extends the maximum payload to 64 bytes and raises the data-phase bit rate beyond 8 Mbit/s while preserving the CAN Classic arbitration and fault-confinement architecture - making it the natural migration path for Everllence's existing CAN infrastructure.

1.2 Existing CAN Controller

The starting point for this project is an existing CAN Classic controller developed internally at Everllence. The controller is implemented in VHDL and has been integrated into a production IO-extender FPGA design. It supports CAN Classic frames with both 11-bit (base) and 29-bit (extended) identifiers at bit rates up to 500 kbit/s, and has been verified through hardware bring-up on physical CAN buses. The initial version was developed by the author of the present document during an internship at Everllence and has since been extensively modified by other engineers at the company.

The controller follows a monolithic architecture: a single top-level wrapper instantiates a combined TX/RX frame FSM, a bit timing generator, a dynamic bit stuffer, a CRC-15 engine, and two Avalon-ST converters for frame serialization and deserialization.

The central component is an 18-state FSM that handles both transmission and reception in a single process. It manages frame arbitration, bit-level TX and RX, stuff-bit error checking, CRC validation, ACK handling, and error flag generation. Error counting (TEC/REC) and node state transitions (error active, error passive, bus off) are handled in separate processes but are tightly coupled to the main FSM through shared signals.

1.2.1 Limitations

While the existing controller is functional for CAN Classic, several areas of the existing design would require significant rework to support CAN FD. Readers unfamiliar with the protocol details referenced below are referred to Section 4.

Single bit rate domain. The bit timing generator produces a single pair of timing strobes derived from a fixed set of bit timing parameters. CAN FD requires switching between a nominal bit rate (used during

arbitration) and a faster data bit rate (used during the data phase), with Transmitter Delay Compensation (TDC) to account for the transceiver round-trip delay at the higher rate. Adding dual bit rate support and TDC would require a fundamental redesign of the timing architecture.

Dynamic bit stuffing only. The bit stuffer implements the CAN Classic rule of inserting an inverse bit after five consecutive identical bits. CAN FD introduces a second stuffing mode - fixed bit stuffing - where a stuff bit is inserted at fixed intervals during the CRC field, and a Stuff Bit Count (SBC) field with Gray-coded parity is appended. The existing stuffer has no mechanism for mode switching or SBC generation.

Single CRC polynomial. The controller uses a single CRC-15 instance. CAN FD requires three CRC polynomials: CRC-15 for Classic frames, CRC-17 for FD frames with payloads up to 16 bytes, and CRC-21 for larger FD payloads. Furthermore, the CRC data feed differs between Classic and FD - in FD frames, dynamic stuff bits in the arbitration region are included in the CRC computation, requiring a dual data feed to the CRC engine.

Coupled fault confinement. Error counting (TEC/REC) and node state transitions are implemented in the same source file as the frame FSM, with no clean sub-layer boundary between them. ISO 11898-1 defines fault confinement as a distinct cross-cutting entity, and separating it as an independently testable module both conforms closer to the standard's reference model and simplifies verification - a fault confinement unit can be exercised in isolation without driving full frame sequences through the FSM.

1.2.2 Decision to Redesign

The rework required across bit timing, bit stuffing, CRC, and frame format complexity - supporting four frame variants with FD-specific control fields across both TX and RX paths - is large enough to justify a clean-slate redesign structured around the ISO 11898-1 layered architecture (LLC, MAC, PCS, FCE) with independently testable subcomponents, rather than retrofitting FD support onto a design not originally built with those boundaries.

1.3 Existing CAN FD IP Cores

Before committing to an in-house redesign, the available CAN FD controller IP cores were evaluated. Table 2 summarizes the candidates, spanning both open-source and commercial offerings.

1.3.1 Open-Source Implementations

CTU CAN FD [1] is the only mature open-source CAN FD controller available as synthesizable HDL. Developed at the Czech Technical University in Prague, it is written in VHDL, licensed under MIT, and has been conformance-tested against ISO 16845-1 [2]. The controller includes a full TX and RX pipeline with up to four TX buffers, acceptance filtering, timestamping, and a register interface with DMA support. A main-line Linux kernel driver has been available since kernel version 5.12. CTU CAN FD represents a complete, production-oriented CAN node - a significantly broader scope than what is needed in this project.

1.3.2 Commercial Implementations

Bosch M_CAN [3] is the reference CAN FD controller, developed by the inventor of both CAN and CAN FD. M_CAN is the IP core embedded in virtually every automotive microcontroller (NXP S32, Infineon AURIX, STM32, TI Jacinto, Renesas RH850). It is licensed under a non-disclosure agreement with per-design royalty fees.

AMD/Xilinx CAN FD [4] is a soft IP core included in the Vivado Design Suite. It provides an AXI4-Lite register interface with up to 32 acceptance filters, TX mailboxes, and RX FIFOs. It is device-locked to AMD/Xilinx FPGAs and cannot be ported to other targets.

CAST CAN FD [5] is a technology-independent RTL core with APB/AHB interface options. It is licensed per-design with an upfront fee. Synopsys (DesignWare) and Cadence offer similar ASIC-targeted CAN FD cores under their respective IP licensing programs.

Table 2: Survey of available CAN FD controller IP cores.

Implementation	Language	License	Scope	Conformance Tested
CTU CAN FD [1]	VHDL	MIT	Full node (TX+RX, buffers, DMA)	ISO 16845-1
Bosch M_CAN [3]	HDL (non-disclosure agreement)	Per-design royalty	Full node	Yes (reference)
AMD/Xilinx CAN FD [4]	HDL	Vivado-included	Full node	Yes
CAST CAN FD [5]	RTL	Per-design fee	Full node	Yes

1.3.3 Rationale for In-House Development

None of these solutions satisfies Everllence’s combined requirements for safety-critical marine engine control. The disqualifying factors span IP ownership, verification authority, architectural scope, interface compatibility, and platform independence - each addressed in turn below.

IP ownership and supply chain independence. Everllence’s engine controllers carry service commitments of up to thirty years. Commercial IP cores introduce a licensing dependency on an external vendor over that full horizon - vendors may discontinue support, change licensing terms, or be acquired. Owning the RTL outright eliminates this exposure and ensures that the design can be maintained, ported, and modified without third-party approval for the full product lifetime. The open-source CTU CAN FD avoids the licensing risk, but using it still means adopting a codebase whose architecture, naming conventions, and design decisions were made for a different context.

Verification authority. In safety-critical domains, the verification evidence must be traceable from standard requirements to RTL assertions and testbench results. Adopting a third-party core - even one conformance-tested against ISO 16845-1 - means inheriting its verification artifacts rather than producing them. Everllence’s verification methodology requires full control over the verification plan, the testbench architecture, and the assertion coverage. Building the RTL in-house allows the verification plan (described in Section 3) to drive the implementation, ensuring that every module is verified against the specific requirements extracted from the standard, using Everllence’s own toolchain and conventions.

Architectural scope. All available IP cores implement a complete CAN node: TX and RX pipelines, message memory, acceptance filtering, buffer management, register interfaces, and in some cases DMA controllers. Everllence’s application requires only the data link layer (LLC, MAC, PCS, FCE) - the protocol engine that converts between byte-level frame data and the serial bus. The higher-level buffering and filtering logic already exists in Everllence’s FPGA infrastructure. Adopting a full-node IP core would introduce unnecessary complexity and area overhead, and the integration effort to bypass or disable the unused subsystems may approach the effort of a targeted implementation.

Integration with existing infrastructure. Everllence’s FPGA designs use a specific Avalon-ST streaming interface for inter-module communication, a particular clock and reset architecture, and established conventions for signal naming and module boundaries. A third-party core would require an adaptation layer to bridge its native interface (Advanced eXtensible Interface, Advanced Peripheral Bus, or custom register map) to the existing infrastructure. The in-house design uses Everllence’s interface conventions natively, eliminating this integration overhead.

Platform independence. The AMD/Xilinx CAN FD core is locked to Xilinx devices. The Bosch M_CAN and other commercial cores are delivered as technology-specific netlists or encrypted RTL for a particular target. The in-house design is written in portable VHDL-93, synthesizable on any FPGA platform or ASIC process flow, ensuring that the IP remains usable if Everllence changes FPGA vendors.

1.4 Problem Statement

The architectural limitations of the existing controller (Section 1.2.1) and the unsuitability of available third-party IP cores (Section 1.3.3) together motivate a clean-slate CAN FD protocol controller conforming to ISO 11898-1. No existing solution combines full IP ownership, a targeted data-link-layer scope matching Everllence’s integration requirements, and native compatibility with Everllence’s Avalon-ST interface conventions and VHDL Code Standard. The design described in this report addresses that gap directly.

1.5 Objectives

- Implement a CAN/CAN FD protocol controller in VHDL, compliant with ISO 11898-1 [6] and supporting the CB, CE, FB, and FE frame formats.
- Structure the design around the ISO 11898-1 sub-layer model (LLC, MAC, PCS, FCE) to enable independent module-level verification.
- Derive and verify a structured set of requirements with traceability from ISO 11898-1 to testbench results.
- Produce an RTL design integrated via Avalon-ST interfaces into Everllence’s existing FPGA infrastructure.

2 Background

This section covers the two technical foundations that the rest of the report builds on. The first is the CAN and CAN FD protocol at the level of motivation and architecture: its bus model, fault-confinement properties, and the bandwidth extensions introduced by CAN FD. The protocol mechanisms referenced by individual requirements - bit timing, stuffing, CRC, and error handling - are covered in depth in Section 4. The second foundation is the VHDL-93 and OSVVM toolchain used for RTL implementation and simulation. Readers already familiar with both may proceed directly to Section 3.

2.1 CAN as a Communication Bus

The Controller Area Network (CAN) is a serial communication bus developed by Bosch in 1986 [7] to connect electronic control units in automotive environments without a central host computer. Where point-to-point wiring and star-switched architectures require a dedicated conductor between every communicating pair, CAN uses a shared two-wire differential bus on which all nodes broadcast simultaneously and arbitrate access without any designated bus master. Any node may initiate a transmission at any time. Contention

is resolved by a non-destructive bitwise arbitration in which the transmitter with the lower-priority identifier detects the collision and silently withdraws, leaving the winner's frame intact. Differential signaling on a twisted pair (ISO 11898-2 physical layer) provides strong common-mode noise rejection - a practical necessity in the electrically harsh environment of an engine bay or industrial cabinet (Figure 1).

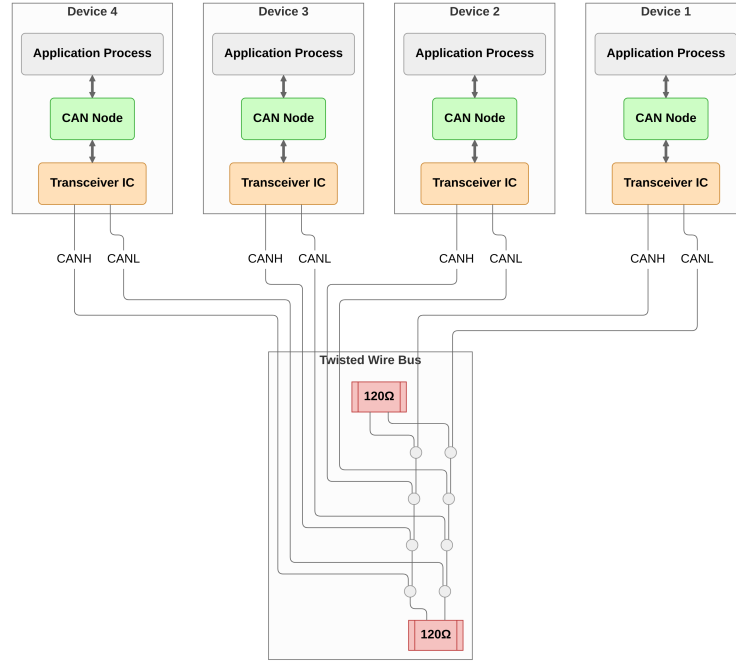


Figure 1: CAN bus with four nodes on a shared differential two-wire bus.

CAN's error-handling architecture is a distinguishing feature relative to simpler serial protocols. Five complementary error detection mechanisms operate concurrently on every transmitted frame: bit monitoring, frame format checking, cyclic redundancy checking, acknowledgment checking, and bit stuffing violation detection. Charzinski showed that under a two-state channel model the residual error probability for an eight-byte frame in a ten-node network is bounded by approximately $3.5 \times 10^{-9} \cdot q_{\text{bad}}$ per frame, where q_{bad} is the probability of a frame being transmitted during a bad channel period [8]. This figure is several orders of magnitude lower than contemporary automotive bus alternatives such as VAN and SCP evaluated under the same model [8]. A fault confinement mechanism tracks each node's error history and automatically escalates from error active through error passive to bus off, electrically isolating a persistently faulty node from the bus without disrupting communication between healthy nodes. Together these properties made CAN the protocol of choice for safety-relevant in-vehicle networks. Adoption subsequently spread to industrial automation, medical devices, and aerospace ground support equipment.

Charzinski's analysis explicitly benchmarked CAN against the contemporary alternatives VAN and SCP under the same channel model, finding CAN's residual error probability to be several orders of magnitude lower - a result he attributed directly to the combination of five concurrent detection mechanisms and the fault confinement escalation model [8]. The fault-confinement and multi-master properties that distinguished CAN from its contemporaries were deliberately preserved in CAN FD: Hartwich's design goal was to extend payload and bit rate while leaving the arbitration and fault-confinement architecture unchanged, specifically to retain the properties that made CAN suitable for safety-relevant distributed control networks [9].

CAN's original data payload was capped at eight bytes per frame, limiting raw throughput to around 1 Mbit/s. As embedded control applications became more data-intensive, this ceiling became a practical con-

straint. CAN FD (Flexible Data Rate), introduced by Bosch in 2012 [9] and incorporated into ISO 11898-1 in 2015, extends the maximum payload to 64 bytes and introduces a separate higher-speed data phase with bit rates of 8 Mbit/s or beyond, while preserving the CAN Classic arbitration phase and the fault-confinement architecture unchanged. The bandwidth constraint imposed by arbitration - where signal propagation time between all nodes limits the bit rate - applies only during the arbitration phase when multiple nodes may simultaneously drive the bus. Once arbitration is resolved and a single transmitter controls the bus, the bit rate can be increased freely, limited only by transceiver slew rate and oscillator stability [9]. Hartwich demonstrated average data rates of 2.5 Mbit/s achievable with standard CAN transceivers, matching the effective payload of a low-speed FlexRay network [9]. At high data-phase bit rates the transceiver's TX-to-RX loop delay (up to 255 ns per ISO 11898-5) may exceed one bit time, requiring Transmitter Delay Compensation (TDC) to correctly position the secondary sample point used for bit-error monitoring. Mutter showed that for data-phase to arbitration-phase bit rate ratios below approximately 9, CAN FD accepts the same oscillator tolerance as CAN Classic, preserving compatibility with commodity crystal oscillators [10].

CAN FD also strengthens the error detection architecture. The longer payloads require stronger CRC polynomials: a 17-bit BCH polynomial covers frames up to 16 data bytes and a 21-bit polynomial covers frames up to 64 bytes, both maintaining Hamming distance 6 [9]. A known weakness in CAN Classic, where two bit errors that generate and eliminate stuff conditions can pass undetected through the CRC [8], is addressed in CAN FD by including dynamic stuff bits in the CRC data feed and introducing the Stuff Bit Count (SBC) field. These improvements together reduce the residual error probability in the worst-case error class by several orders of magnitude compared to CAN Classic [11]. The governing standard for this project is ISO 11898-1 [6], which specifies both CAN Classic and CAN FD data link layer and physical signaling requirements.

2.2 Tools and Language

The RTL source is implemented in VHDL-93. Everllence's synthesis toolchain uses Quartus Prime, which does not fully support VHDL-2008 constructs in synthesis, making VHDL-93 the practical upper bound for synthesizable RTL. Testbenches are written in VHDL-2008 to support the OSVVM verification framework [12], which requires VHDL-2008 language features. SystemVerilog with UVM is the dominant industry alternative for RTL implementation and verification at this scale. The choice here follows company convention rather than a project-level technical comparison. GHDL [13] is used for simulation and supports both language revisions.

3 Requirements

This section defines what must be implemented and verified. The Everllence coding constraints establish the implementation framework - port type restrictions, naming conventions, and testbench structure - that applies uniformly to all in-house FPGA modules. The 38 protocol requirements are derived from ISO 11898-1 normative obligations through an AI-augmented extraction pipeline and distilled manually into independently verifiable entries, each carrying a source clause reference, a priority rating, and a verification method. Together they bound the design space before any architectural decisions are made.

3.1 VHDL Code Standard and Design Constraints

The constraints on this project come from two distinct sources. Two are specific to this project's integration context: the Avalon-ST host interface and the mandatory use of the IP library CRC block. The remainder

are drawn from Everllence’s VHDL Code Standard and apply uniformly to all in-house FPGA modules.

3.1.1 Project-specific Infrastructure Requirements

1. **Avalon-ST user interface:** The CAN controller’s external interface to the host system must use the Avalon-ST streaming protocol [14] (data, valid, ready, sop, eop). The requirement applies specifically to the boundary between the CAN controller and its user.
2. **IP library CRC block:** Everllence maintains a reusable, parameterised CRC generator (`gen_crc`) in its IP library. This module must be used for all CRC computations.

3.1.2 File Structure and Naming

1. **Per-module file structure:** Each module must be organized into a fixed directory layout: `hdl_src/` for RTL source files, `hdl_tb/` for testbench files, and `test_case/` for waveform configuration. One entity per VHDL file, with the filename matching the entity name. Every entity requires a dedicated testbench, unless it is instantiated exclusively as a submodule of a fully tested parent.
2. **Entity port types:** Entity ports are restricted to `std_logic`, `std_logic_vector`, and records or arrays of these types. Modes are restricted to `in` and `out`.
3. **Naming conventions:** A mandatory prefix/suffix scheme applies to all VHDL identifiers: types (`t_`), constants (`c_`), generics (`gc_`), processes (`p_`), functions (`f_`), packages (`pk_`), state variables (`s_`), entity inputs (`_i`), and entity outputs (`_o`).

3.1.3 RTL Coding Style and Verification

1. **RTL design rules:** Synchronous processes must be sensitive to the clock only. Reset must be synchronous and initialize all control registers. FSMs are preferably implemented as single-process designs where all signal assignments are derived from the current state, with an explicit other-state that returns to a known safe state.
2. **Testbench requirements:** Testbenches must follow a black-box testing model and test cases must be ordered as: reset tests first, then a normal-usage test, then all remaining tests.

3.2 From Specification to Structured Requirements

The requirements engineering process addressed two key objectives [15]:

1. Extracting a clear and actionable set of requirements that could serve as a starting point for the design phase.
2. Establishing a clear, traceable link between the ISO 11898-1 specification and the verification environment.

Both objectives are complicated by the source material: normative requirements are distributed across subsections, often restated from different perspectives, and interspersed with explanatory text. The standard compounds this by bundling multiple obligations into single clauses, interspersing normative `shall` statements with informative rationale prose, and repeating equivalent obligations from both transmitter and receiver perspectives.

The AI-augmented pipeline shown in Figure 2 was designed to address these extraction challenges systematically. The first step was converting the ISO 11898-1 PDF to Markdown - a format that can be efficiently searched and ingested by LLMs. The resulting Markdown file was then fed to a Claude Sonnet 4.6 LLM

agent, which was prompted to extract all normative statements - sentences containing words like “shall”, “should”, “must”, and their corresponding negations.

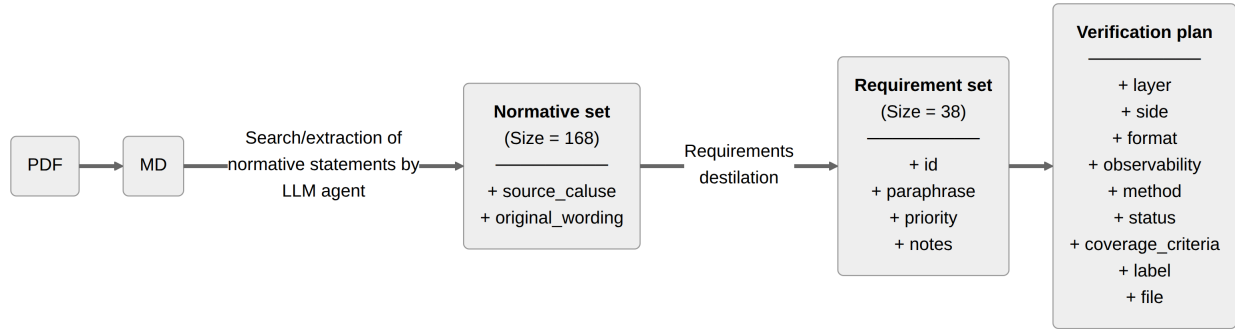


Figure 2: Pipeline generating the `verification_plan.toml` artifact from the ISO 11898-1 standard.

This process yielded a raw set of 168 normative statements linked to the ISO standard sections from which they were extracted. The normative set was then manually reviewed, consolidated, and distilled into a final set of 38 requirements (reproduced in Section A).

The requirements set is stored as a configuration file (`verification_plan.toml`), one `[[requirement]]` block per entry. To support ongoing AI-assisted refinement without the risk of silent data corruption, a custom Model Context Protocol server (`verification_plan_manager.py`) was developed alongside the plan. The server exposes query, update, insert, and statistics operations as tool calls that the AI coding agent can invoke directly within its development environment. Each write operation targets a single requirement entry and validates field values against the schema before committing. This makes it structurally impossible for the agent to silently drop entries, fabricate field values, or corrupt the file syntax - failure modes that arise inevitably when an LLM is asked to rewrite a large structured file in one operation. During the requirements phase, the agent worked exclusively with the five fields relevant at this stage:

- **source_clause:** The ISO 11898-1 section reference (e.g. §6.6.13.1). This is the traceability anchor - every requirement links back to the clause from which it was distilled, making it possible to verify the requirements set against the standard during review.
- **original_wording:** Verbatim ISO text for the relevant clauses. Preserving the source wording prevents paraphrase drift and provides a fallback for resolving ambiguity during implementation.
- **paraphrase:** A concise, implementer-facing restatement of the requirement. Where the ISO prose bundles multiple obligations into a single clause, the paraphrase enumerates them as numbered sub-claims, each independently verifiable.
- **priority:** A three-level rating - P1 (need-to-have, derived from “shall” obligations and core correctness), P2 (verified in the normal cycle), or P3 (optional, derived from “should” clauses or implementation-dependent features). Priority drives implementation sequencing and scope decisions when schedule is constrained. Of the 38 requirements, 31 are rated P1, five are P2, and two are P3. Every requirement not rated P1 was explicitly demoted. The rationale for each demotion is given in Table 3.
- **notes:** Residual clarifications not resolved by the paraphrase - implementation constraints, out-of-scope markers, or known ambiguities flagged for design review.

Table 3: Priority demotion rationale for all requirements not rated P1.

ID	Topic	Priority	Demotion rationale
REQ-002	LLC TX request and abort timing	P2	The 2-SOF processing window is a responsiveness guarantee, not a correctness constraint. A node that transmits eventually but outside this window sends valid frames.
REQ-011	Remote frame	P2	A data-only node is a valid CAN implementation. Remote frame support is a distinct feature subset not required for basic interoperability.
REQ-016	ESI bit transmission	P2	ESI communicates the node’s error state as an informational signal. Incorrect ESI does not abort a frame or trigger a protocol error at any receiver.
REQ-036	MAC data consistency	P2	A frame corrupted mid-transmission would be error-flagged by other nodes on the bus and retransmitted. The requirement reduces bus pollution from invalid frames but the node remains functional without it.
REQ-037	Error signaling enable	P2	Error signaling itself is covered by P1 requirements. This requirement concerns only the existence of a configurable disable mode, which is an optional operational feature.
REQ-004	Frame acceptance filtering	P3	ISO uses advisory “should” language. Acceptance filtering is also an application-layer concern above the LLC boundary verified here.
REQ-038	DLC padding	P3	Padding with 0xCC applies only when the implementation exposes a configurable maximum-data-byte restriction. The feature may be waived entirely if that restriction is not implemented.

3.3 AI-Assisted Extraction: Utility and Limitations

The AI-assisted workflow delivered value in two distinct phases of the project, with very different characteristics in each.

In the extraction phase, the LLM agent earned its keep by bootstrapping and linking the initial normative statement set. Having a fully populated and linked starting point - even one requiring substantial revision - gave the manual review process a concrete artifact to work from. The time saving from the extraction itself was, however, marginal. The agent’s output had to be reviewed statement by statement, which is functionally similar to extracting requirements manually in the first place. The primary benefit of the AI-assisted approach in this phase was therefore not efficiency, but rather the increased consistency of an automated pass over the full standard text.

The distillation step that followed - consolidating 168 raw normative statements into 38 prioritized, independently verifiable requirements - required substantial manual effort that the AI could not replace. Deciding which statements address the same underlying obligation, how to bound each requirement so that it is independently verifiable, and how to assign priority in a way that is defensible against the standard text are judgment calls that depend on understanding the protocol at an implementation level.

The MCP server interface proved genuinely useful throughout the design, implementation, and verification phases that followed extraction. As implementation decisions were made, requirements were refined - phrases sharpened, notes extended, observability classifications updated, and traceability fields populated.

The AI coding agent performed these updates directly through the MCP tool calls, targeting individual requirement fields against a schema-validated store. The alternative - asking an agent to rewrite the full TOML file each time a requirement changed - would have introduced silent data corruption risks at every edit. The narrow, validated write interface made incremental AI-assisted maintenance of the verification plan safe and practical across all three project phases.

The 38 requirements, each linked to its ISO source clause and assigned a priority, define the scope of what must be implemented and verified. They also function as a structured map to the protocol: every requirement points to a mechanism that must be understood before implementation can begin. Section 4 provides that understanding - covering the sub-layer model, frame formats, bit timing, bit stuffing, CRC, and error handling.

4 CAN and CAN FD Protocol Overview

Each requirement in Section 3 refers to a specific protocol mechanism. This section covers those mechanisms - the sub-layer model, frame formats, bit timing, stuffing, CRC, and error handling - each cross-referenced to the relevant REQ-NNN entries. Readers familiar with ISO 11898-1 may skip to Section 5.

4.1 Layered Reference Model

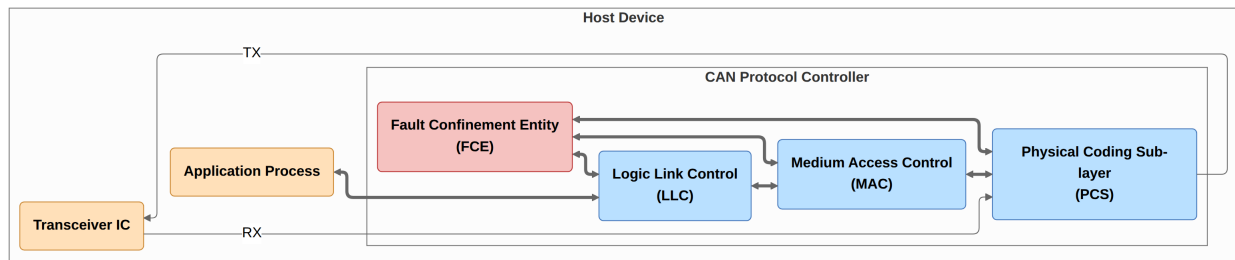


Figure 3: ISO 11898-1 CAN node reference model showing the LLC, MAC, PCS sub-layers and cross-cutting FCE.

ISO 11898-1 structures the CAN data link layer into three functional sub-layers and a cross-cutting Fault Confinement Entity (FCE) (Figure 3):

- **LLC (Logical Link Control)**: accepts frame requests from the host application, applies retransmission policy on error or lost arbitration, and supplies frames to the MAC in serialized form.
- **MAC (Medium Access Control)**: encodes and decodes the frame bit-by-bit - performing bit stuffing and destuffing, CRC generation and checking, and acknowledgment handling - and governs bus access arbitration.
- **PCS (Physical Coding Sublayer)**: manages bit timing, clock synchronization (including Transmitter Delay Compensation for FD data phase), and the sample/drive interface to the physical transceiver.
- **FCE (Fault Confinement Entity)**: maintains Transmit Error Counter (TEC) and Receive Error Counter (REC), escalating the node's error state from error active through error passive to bus off as error counts accumulate.

In the implementation described in this report, each sub-layer maps to a dedicated VHDL module, and the sub-layer interfaces become the port records connecting those modules (Section 6). LLC service obligations are captured in REQ-001 through REQ-005 and REQ-033. MAC frame-encoding rules are in REQ-006 through REQ-024 and REQ-032 through REQ-038. PCS timing constraints are in REQ-025 through REQ-028.

FCE counter and state-transition rules are in [REQ-029](#) through [REQ-031](#) and [REQ-037](#).

4.2 Frame Types and Formats

CAN defines two classes of frames: CAN Classic (CC) and CAN FD (FD). Within each class, frames may carry either an 11-bit base identifier or a 29-bit extended identifier, giving four frame formats: CB (Classic Base), CE (Classic Extended), FB (FD Base), and FE (FD Extended), as shown in Figure 4. Classic frames (CB and CE) additionally support remote frame variants (RTR=1, no data field), giving six bus frame types in total. CAN XL frames are out of scope for this project.

A CAN Classic frame consists of Start of Frame (SOF), Arbitration field (identifier, RTR, IDE), Control field (DLC), Data field, CRC field, ACK slot and delimiter, End of Frame (EOF), and Intermission. SOF is a single dominant bit that marks the beginning of a frame and triggers hard synchronization in all receiving nodes ([REQ-010](#)). Within the arbitration field, bits are transmitted MSB first ([REQ-034](#)). The RTR bit distinguishes data frames from remote frames, being dominant for data frames and recessive for remote frames ([REQ-011](#)). In extended frames, an SRR placeholder bit transmitted recessive precedes the IDE bit ([REQ-012](#)). The DLC encodes the number of data bytes using the four-bit mapping defined in ISO 11898-1 ([REQ-033](#), [REQ-038](#)). After the data and CRC fields, the ACK slot carries a dominant bit driven by every receiver that has successfully validated the frame CRC - the transmitter monitors this slot and reports an acknowledgment error if no dominant bit is received ([REQ-014](#)). The frame is delimited by seven recessive EOF bits followed by three recessive intermission bits ([REQ-020](#), [REQ-008](#)).

A CAN FD frame shares the same structure through the arbitration phase and then introduces FD-specific control fields. The FDF bit distinguishes an FD frame from a Classic frame. A recessive FDF triggers the FD control field sequence including reserved bits, BRS, and ESI ([REQ-015](#)). The BRS (Bit Rate Switch) bit controls the transition to the data-phase bit rate: when BRS is recessive the bus switches to the faster data rate immediately after the BRS sample point and returns to the nominal rate at the CRC delimiter ([REQ-032](#)). The ESI (Error State Indicator) bit reflects the transmitting node's fault-confinement state: a node in error passive state shall transmit ESI recessive ([REQ-016](#)).

4.3 Bit Timing and Flexible Data Rate

Every CAN bit period is divided into four non-overlapping time segments measured in Time Quanta (TQ), where one TQ equals the period of the prescaled system clock (Figure 5). CAN FD extends CAN Classic with an independently configured data-phase bit rate. A CAN FD node therefore maintains two independent sets of segment parameters, one for the nominal rate and one for the data rate ([REQ-025](#)):

- **Sync Segment (SYNC_SEG):** one TQ. The point at which the bus is expected to produce a recessive-to-dominant edge after synchronization.
- **Propagation Segment (PROP_SEG):** compensates for round-trip signal propagation delay on the bus and in the transceiver. It shall be programmed to be at least as long as twice the maximum bus propagation delay.
- **Phase Segment 1 (PHASE_SEG1):** immediately precedes the sample point. Can be lengthened by the resynchronization mechanism to absorb positive phase errors.
- **Phase Segment 2 (PHASE_SEG2):** follows the sample point to the end of the bit. Can be shortened to absorb negative phase errors.

The **sample point** falls at the PHASE_SEG1 / PHASE_SEG2 boundary. Every receiver samples the bus exactly once per bit at this point. The sample point position - expressed as a percentage of the total bit

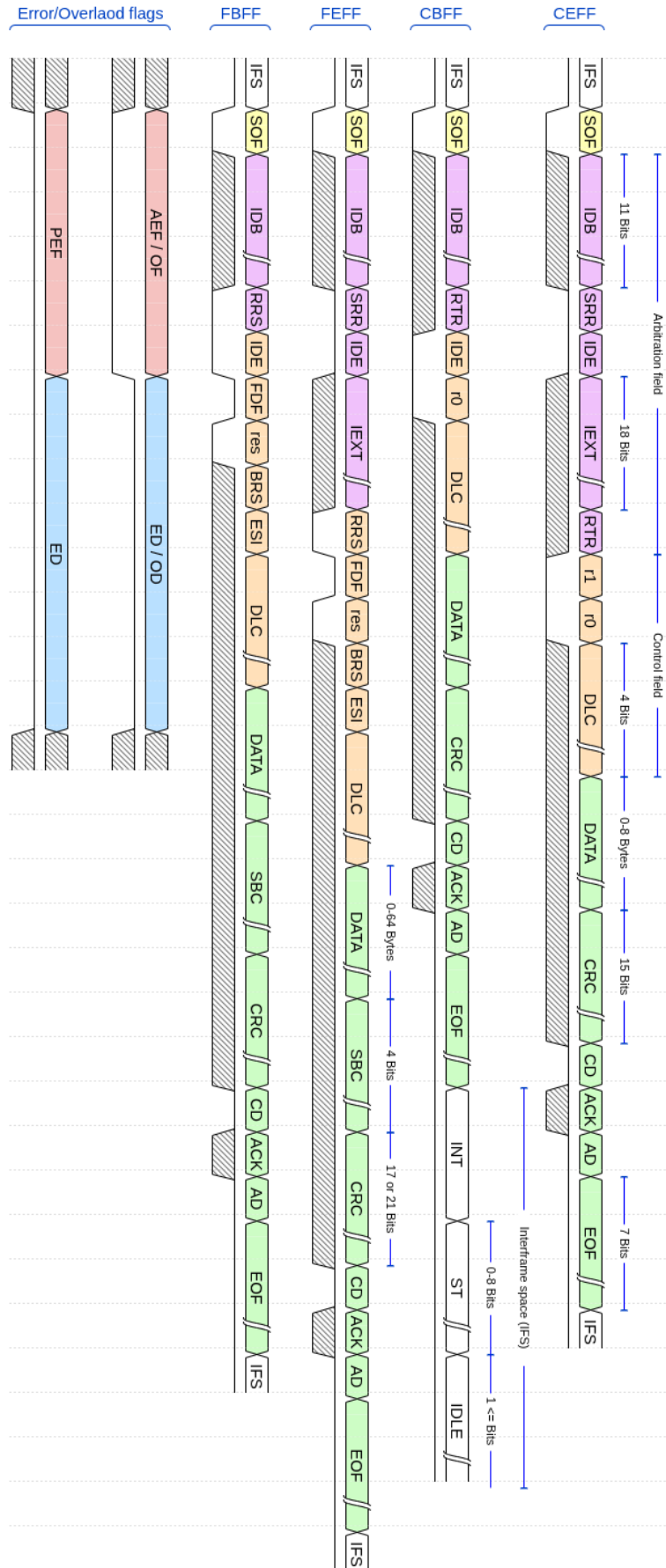


Figure 4: Frame formats for the four in-scope frame types (CB, CE, FB, FE) and the error and overload flags. Field widths are annotated per ISO 11898-1.

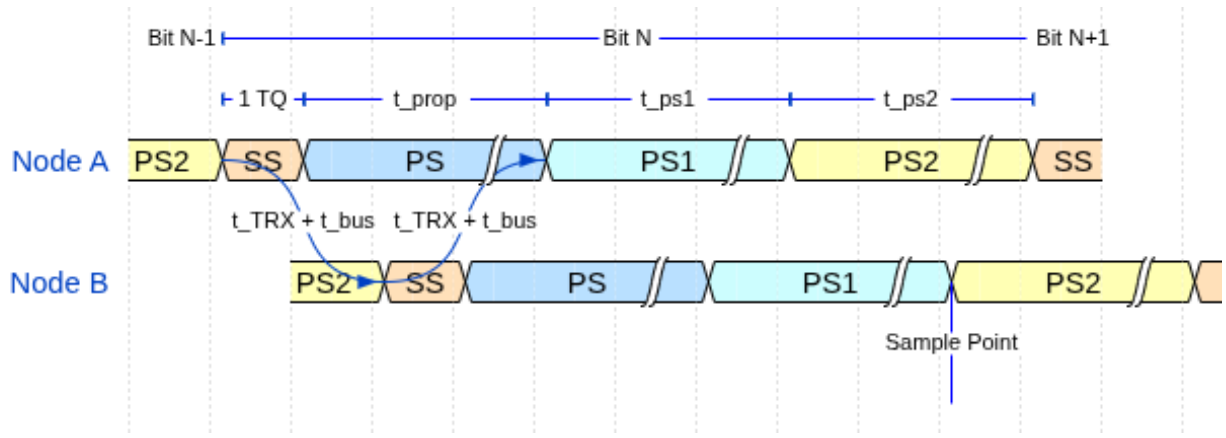


Figure 5: CAN bit time structure showing the four segments (SS, PS, PS1, PS2), the sample point, and propagation delays on a two-node bus.

time - is a configuration parameter traded off against bus length, node count, and oscillator tolerance.

Resynchronization corrects for accumulated phase error between a receiver's local oscillator and the transmitter's bus edges. Hard synchronization forces a full re-alignment on the SOF falling edge at the start of each frame. During the frame, resynchronization adjusts PHASE_SEG1 or PHASE_SEG2 by up to the configured Synchronization Jump Width (SJW) on each recessive-to-dominant edge, keeping the sample point aligned with the transmitter. Only one synchronization is permitted within a single bit time (REQ-027). The two resynchronization cases are illustrated in Figure 6.

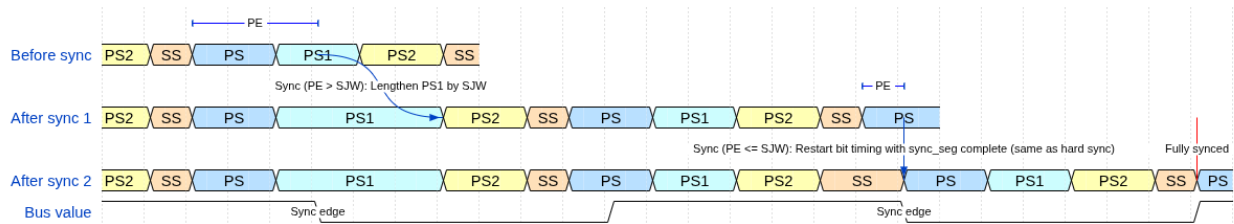


Figure 6: Resynchronization over two successive sync edges. PE is the phase error relative to SYNC_SEG. SJW is the Synchronization Jump Width.

CAN FD and the flexible data rate. CAN FD introduces a second, independently configured bit rate for the data phase. The BRS (Bit Rate Switch) bit in the FD control field (Figure 4) controls this transition: when BRS is recessive, the bus switches to the data-phase bit rate immediately after the BRS sample point and returns to the nominal rate at the CRC delimiter. The nominal rate governs the arbitration phase (SOF through BRS) and the return path (CRC delimiter onward). The data rate governs the payload and CRC fields in between. Because the data phase operates at a much shorter bit time, the same physical propagation delay represents a larger fraction of the bit period. On electrically long buses at high data rates, the loop propagation delay can exceed a full data-phase bit time.

Transmitter Delay Compensation (TDC) addresses this. A transmitter in the FD data phase cannot rely on immediate bus loopback for bit-error monitoring, because the echo of a driven bit arrives one or more bit times late. TDC measures the actual round-trip delay at the start of the data phase and configures a Secondary Sample Point (SSP) at the correct offset, so that each transmitted bit is still checked for loopback correctness. The TDC measurement and SSP configuration are PCS responsibilities and are a significant driver of PCS complexity in the implementation (Section 7.7, Figure 7).

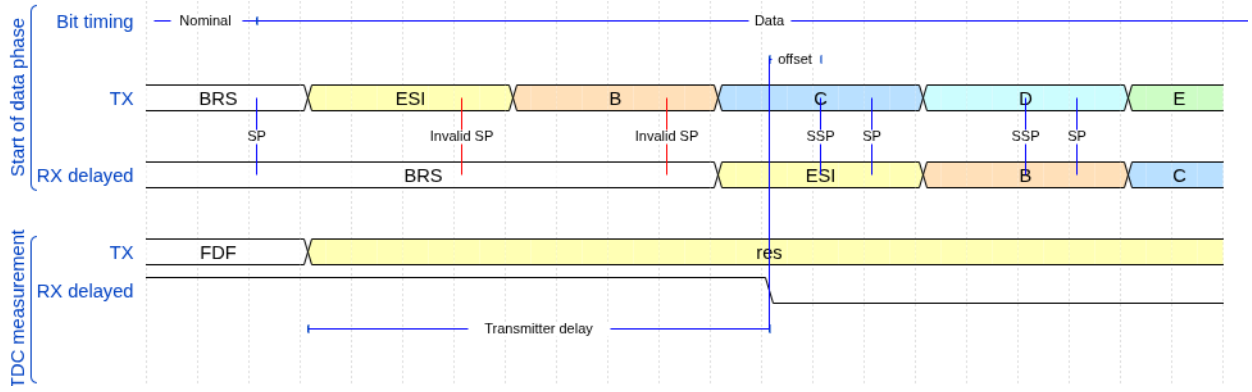


Figure 7: Transmitter Delay Compensation (TDC). The loop delay t_{loop} is measured on the first data-phase bit and used to position the SSP at $t_{\text{SSP}} = t_{\text{TDC_offset}} + t_{\text{measured}}$.

4.4 Bit Stuffing

Bit stuffing ensures sufficient transitions on the bus for receiver clock synchronization. CAN Classic applies dynamic stuffing throughout the frame: after five consecutive bits of the same polarity, the transmitter inserts one complement stuff bit and the receiver removes it before forwarding the data stream (REQ-019). CAN FD retains dynamic stuffing through the arbitration phase, then switches to a combined dynamic-plus-fixed scheme in the data phase. Fixed stuff bits are inserted at predetermined positions (every fourth bit in the CRC field, independent of the preceding bit pattern). They carry a parity-encoded Stuff Bit Count (SBC) field that allows receivers to independently verify the number of dynamic stuff bits seen in the frame - an additional error detection layer absent in CAN Classic (REQ-017, Figure 8).

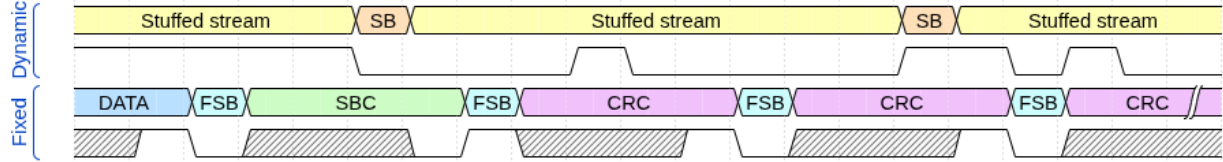


Figure 8: Dynamic and fixed bit-stuffing examples showing stuff bit placement for both encoding modes.

4.5 Cyclic Redundancy Check

The CRC polynomial and field length depend on frame type and data payload length (REQ-006):

- **CRC-15:** used for all CAN Classic frames. Dynamic stuff bits are excluded from the CC CRC computation - the CRC accumulates over the destuffed bit stream from SOF through the end of the data field (REQ-013).
- **CRC-17:** used for FD frames with data payloads up to 16 bytes (DLC 0-10).
- **CRC-21:** used for FD frames with data payloads from 20 to 64 bytes (DLC 11-15).

The key asymmetry between CC and FD concerns the CRC data feed. For FD frames, dynamic stuff bits within the arbitration region are included in the CRC computation, while for CC frames they are excluded (REQ-013). Fixed stuff bits in the FD CRC region are also included. This dual data-feed requirement is a direct consequence of REQ-013 and has concrete consequences for the MAC implementation described in Section 7.5. The CRC field is terminated by a recessive CRC delimiter bit (REQ-018). A received frame with a CRC mismatch causes the detecting node to transmit an error flag.

4.6 Error Detection and Fault Confinement

Every CAN node monitors the bus for five categories of error ([REQ-022](#)). Bit errors occur when a transmitter reads back a polarity different from what it drove. Stuff errors occur when six consecutive bits of the same polarity appear where the stuffing rule prohibits it. CRC errors occur when the received checksum does not match the locally recomputed value. Form errors occur when fixed-format fields contain illegal bit values. Acknowledgment errors occur when a transmitter receives no dominant ACK bit from any receiver ([REQ-014](#)). Detection of any of these errors causes the detecting node to immediately transmit an error flag, aborting the in-progress frame ([REQ-023](#)). An error active node transmits an active error flag consisting of six consecutive dominant bits. An error passive node transmits a passive error flag of six consecutive recessive bits instead ([REQ-007](#)). Both are followed by an eight-bit recessive error delimiter.

The FCE tracks each node's error history through TEC and REC. Counter increments and decrements follow the rules in ISO 11898-1 sec. 8.1.4.2 ([REQ-030](#)). A node begins in error active and transitions to error passive when either counter exceeds 127 ([REQ-031](#)), then to bus off when TEC exceeds 255. In bus off the node ceases all bus activity and shall not influence the bus ([REQ-028](#)) until 128 sequences of 11 consecutive recessive bits are observed, after which TEC and REC are reset and the node returns to error active. A host-initiated `llc_i.normal_mode` assertion also resets the FCE to its initial state immediately ([REQ-029](#)). Whether error signaling is enabled at all is a run-time configuration parameter ([REQ-037](#)). This escalation mechanism is the subject of several verification plan requirements and directly motivates the separation of the FCE into a dedicated module with its own testbench.

With those mechanisms established - sub-layer boundaries, frame formats, bit timing and the dual data rate, stuffing rules, CRC polynomials, and the fault confinement escalation model - Section 5 introduces the five classification dimensions of the verification plan and shows how each one connects back to the protocol concepts described here.

5 Verification Plan

The requirements set established what must be true about the implementation - 38 entries, each naming a protocol obligation and linking it to its ISO clause. But requirements in that form are not yet actionable as verification tasks: they say nothing about which module testbench should exercise them, what stimulus configurations are needed, whether internal signals must be observable, or how completion will be recognized. Turning the requirements set into a verification plan means answering those questions explicitly for each entry, before implementation begins.

The plan was populated through the same Model Context Protocol server introduced in Section 3, which validated each field value against the schema before committing. The five classification dimensions fall into two groups. Three are design-facing - `layer`, `side`, and `format_applicability` - determining where each requirement belongs in the module decomposition and what stimulus configurations its testbench needs. Two are verification-facing - `observability` and `verification_method` - resolving whether a requirement can be checked through port signals or requires access to internal state, and specifying the verification technique. Priority spans both groups, driving implementation sequencing and determining which requirements must be closed before the design is considered complete. The rationale and allowed values for each dimension are described in Section 5.1 through Section 5.7.3, followed by the full verification plan data structure and its traceability fields in Section 5.7.

5.1 Layer

The layer field assigns each requirement to the protocol sub-layer that owns it (LLC, MAC, PCS, or FCE - see Section 4.1), determining the verification boundary at which the requirement must be exercised. A fifth label - **system** - classifies requirements that are inherently multi-layer or multi-node in character. Some CAN behaviors cannot be attributed to a single layer of a single node: they emerge from interactions between multiple nodes on the bus, or span the layer boundary within a single node. The system label flags these requirements as ones that require either an integrated multi-module testbench or a multi-node simulation environment.

At design time, this classification directly motivated the layered module architecture: requirements assigned to a given layer pointed to the corresponding module as the responsible implementation unit and to that module's testbench as the primary verification environment. The consequence of the system label is described further in Section 6.2.2.

5.2 Side

The side field records whether a requirement pertains to the transmitter path, the receiver path, or both roles simultaneously. This dimension reflects the ISO standard's own framing, which frequently specifies transmitter and receiver obligations separately. In the verification environment, the side field determines whether a testbench drives the DUT in transmitter mode, receiver mode, or both roles in succession within a single test scenario. The design consequences of this dimension - and why it appeared to motivate a split-path architecture but did not - are discussed in Section 6.2.2.

5.3 Format Applicability

The format_applicability field records which of the in-scope frame formats (CB, CE, FB, FE - see Figure 4, where CB and CE implicitly cover remote frame variants) each requirement applies to. Because the formats differ in stuffing mode, CRC polynomial, and control field structure (Section 4), a requirement that applies only to FD frames implies stimulus configurations with FDF=1 and DLC values spanning both the CRC-17 and CRC-21 threshold, while a requirement that applies to all four formats must be exercised across all format-specific configurations. The field makes those implications explicit rather than leaving them to be inferred from the requirement text.

5.4 Observability

The observability field resolves each requirement as either black-box or white-box, relative to the module boundary of the owning layer:

- **Black-box:** Can be verified purely through the module's observable port signals.
- **White-box:** Verification requires direct observation of the module's internal state.

This distinction has direct consequences for testbench architecture. Black-box requirements are verifiable with stimulus-and-observe testbenches that drive inputs and check outputs without any knowledge of internal implementation. White-box requirements - which include CRC polynomial correctness, bit counter arithmetic, error counter thresholds, and Gray-coded SBC encoding - require a parallel reference model that re-computes the expected value independently, or direct observation of internal signals via testbench signal access.

5.5 Priority

The priority field classifies each requirement into one of three levels:

- P1 requirements are need-to-have - they must be verified before the design can be considered complete.
- P2 requirements are nice-to-have - they are verified in the normal verification cycle but do not block closure.
- P3 requirements are optional - addressed only if schedule permits.

The final plan contains 31 P1, five P2, and two P3 requirements. The demotion rationale for each requirement not rated P1 is given in Table 3. Of the 31 P1 requirements, 26 are closed. Four ([REQ-001](#), [REQ-003](#), [REQ-005](#), [REQ-033](#)) remain not started pending can_llc implementation, and [REQ-022](#) is in progress with partial simulation coverage. P2 and P3 requirements are addressed as schedule permits.

5.6 Requirement Distribution

Table 4 shows the 38 requirements distributed across layer and observability. MAC dominates both in count and in white-box density, reflecting the breadth of frame-encoding logic that must be verified against internal bit-level state. FCE requirements are entirely black-box: fault-confinement state transitions are fully observable through the node's error-state output signals without needing access to internal counters. System requirements - those that require a multi-node or multi-module environment - split roughly evenly between the two observability classes.

Table 4: Requirement distribution by layer and observability.

Layer	Black-box	White-box	Total
MAC	3	16	19
LLC	4	3	7
System	2	3	5
PCS	1	3	4
FCE	3	0	3
Total	13	25	38

5.7 Verification Plan Data Structure

The verification plan data structure (Table 5) augments each requirement with the dimensions needed to answer not just *what* must be true, but *how* it will be verified, *where* the evidence lives, and *when* verification is complete. The plan evolved continuously as implementation and verification work progressed. Two dimensions were not part of the initial taxonomy: the `system` layer label was added when it became clear that some CAN behaviors emerge from multi-node interactions and cannot be attributed to any single module's testbench. The `observability` field was introduced when the distinction between black-box and white-box verification had direct consequences for testbench architecture that were not apparent from the requirement text alone. The following sub-sections detail the rationale and allowed values for each field in the verification plan. The complete verification plan is reproduced in Section A as two separate tables (linked by common IDs).

Table 5: Verification-plan data structure fields.

Field	Purpose
<code>id</code>	Sequential identifier REQ-NNN.
<code>source_clause</code>	ISO 11898-1:2015 section reference.
<code>original_wording</code>	Verbatim normative text excerpts from the ISO standard.
<code>paraphrase</code>	Concise paraphrase of the <code>original_wording</code> field (this is the actual requirement)
<code>layer</code>	Sub-layer owner: LLC, MAC, PCS, FCE, or system (Section 5.1).
<code>side</code>	transmitter, receiver, or both (Section 5.2).
<code>format_applicability</code>	Applicable frame formats: CB, CE, FB, FE (Section 5.3).
<code>observability</code>	<code>black_box</code> or <code>white_box</code> (Section 5.4).
<code>verification_method</code>	Method(s) used to verify the requirement (Section 5.7.1).
<code>priority</code>	P1 (need-to-have), P2 (nice-to-have), or P3 (optional) (Section 5.5).
<code>status</code>	<code>not_started</code> , <code>in_progress</code> or <code>complete</code> (Section 5.7.3).
<code>notes</code>	Residual clarifications not resolved by the paraphrase - implementation constraints, out-of-scope markers, or known ambiguities flagged for design review.
<code>label</code>	Assertion label, TB procedure name, coverage ID, or RTL tag. Comma-separated when multiple procedures cover distinct sub-claims (Section 5.7.2).
<code>file</code>	Target file: TB for simulation/coverage, RTL for code inspection. Comma-separated when sub-claims span multiple files (Section 5.7.2).

5.7.1 Verification Method

The `verification_method` field makes the path from requirement to verification artifact explicit and actionable. Four methods are used: `simulation` (automated assertion procedures in a testbench), `code_inspection` (RTL source review), `waveform_inspection` (manual review of simulation output), and `coverage` (a functional coverage bin that records whether a specific condition or value range was exercised during simulation). Combinations are valid when multiple sub-claims within one requirement each call for a different method.

5.7.2 Traceability: Label and File

Each requirement entry carries two dedicated traceability fields: a `file` field identifying the testbench or RTL source file responsible for covering the requirement, and a `label` field identifying a specific named procedure, assertion, or coverage ID within that file. Together they establish a direct, navigable link from each requirement to its verification artifact.

5.7.3 Status

The `status` field (`not_started`, `in_progress`, `complete`) records requirement closure state explicitly, allowing partial progress to be tracked.

The verification plan - 38 requirements each classified along five dimensions and each linked to a testbench file and assertion label - served both roles in what followed. The design-facing dimensions (`layer`, `side`, `format_applicability`) constituted the primary architectural inputs for the design phase, mapping requirements to module boundaries and implementation scope. The verification-facing dimensions (`observability`, `verification_method`, `label`, `file`) defined the testbench architecture and evidence type for each requirement. How

the design-facing dimensions shaped the module decomposition - and where the apparent mapping from requirements structure to design structure broke down - is the subject of Section 6.

6 Design and Architecture

The verification plan classified all 38 requirements along three design-facing dimensions: `layer`, `side`, and `format_applicability`. Two led directly to sound architectural choices. One pointed toward a split TX/RX architecture that was attempted, found unworkable, and replaced by the unified `can_mac_fsm`.

6.1 Ramifications of the Requirements Model on Initial Design Strategy

The structure of the requirements model had direct consequences for the initial design strategy, in ways that were not fully anticipated at the outset.

The **layer dimension** mapped naturally onto the ISO standard's own layered reference model, making a layered module architecture the obvious implementation strategy. A dedicated hardware module for each layer - MAC, LLC, fault confinement, and PCS - would allow requirements pertaining to a given layer to be verified in isolation. The observability dimension reinforced this directly: black-box requirements mapped cleanly onto port-level stimulus and observation, while white-box requirements pointed toward the need for reference models or direct internal signal observation, both of which are most tractable in a per-module testbench. This was a sound conclusion: the modular architecture proved to be the right design choice.

The **TX/RX side dimension** had a subtler and more consequential effect. Organizing requirements along the transmitter/receiver axis made intuitive sense from a specification perspective - the ISO standard itself frames many requirements in terms of transmitter behavior and receiver behavior - and it was genuinely useful for thinking through which requirements belonged where. However, it also made a split-path implementation architecture look like the natural design strategy, simply because the requirements were literally organized along that split. The implication appeared to be: implement a TX module, implement an RX module, and map the TX requirements to the former and the RX requirements to the latter.

This turned out to be a red herring. The pitfalls of the split-path approach were not at all apparent from the requirements table alone. The table made the split architecture look clean and well-motivated. The problems - design drift between separately implemented FSMs, integration complexity, and unnecessary hardware duplication - only surfaced later, during integration. The structure of a requirements model can inadvertently bias architectural decisions in ways that are not immediately obvious, and the apparent naturalness of a design strategy that mirrors the requirements structure is not in itself a reliable signal that the strategy is sound.

The **format_applicability dimension** had a more constructive effect. Requirements tagged with a specific format subset (CB, CE, FB, FE, or combinations) made explicit which frame variants each protocol mechanism must handle, which in turn shaped two concrete design decisions. First, the MAC FSM needed per-field state granularity rather than per-phase granularity: because different format subsets diverge at specific field boundaries (the IDE/FDF/RES sequence differs between Classic and FD, and between base and extended addressing), encoding those branches in per-field states keeps each state's logic narrow and format-specific transitions visible in the state graph. Second, the LLC-to-MAC streaming interface needed a front-loaded metadata layout - all format flags available before the first ID bit is needed - so that the MAC FSM could determine its branch path without buffering the entire frame. Both decisions are detailed in Section 6.2.3 and Section 6.2.4 respectively.

6.2 Architectural Design Decisions

The ramifications identified above narrowed the design space early: a layered architecture was well-motivated by the requirements model, and a unified FSM proved necessary once the split-path approach was attempted. The primary inputs were the existing in-house CAN Classic controller (Section 1.2) and the ISO 11898-1 standard's own layered reference model [6]. CTU CAN FD [1] is noted as an existing open-source CAN FD implementation but was not used as a design reference.

6.2.1 Adopting the ISO 11898-1 Sub-layer Model

The existing controller already has a modular structure: a coordinating frame FSM, a bit stuffer, a CRC engine, a serializer, and a PCS equivalent for bit timing. The functional decomposition is sound. What it lacks is alignment with the ISO 11898-1 sub-layer model - specifically, fault confinement logic (TEC/REC management and node state transitions) is implemented within the main frame FSM process rather than as an independently testable entity.

For CAN Classic with a single bit rate and two addressing variants, this coupling is workable. CAN FD changes the picture: dual bit rates, three CRC polynomials, fixed bit stuffing with SBC encoding, and FD-specific control fields across six bus frame types substantially increase the complexity of every FSM state. With fault confinement embedded in that FSM, isolating and verifying the TEC/REC counter rules becomes difficult - any fault confinement test must also drive a full frame sequence through the FSM to exercise it.

The approach adopted here is to follow the ISO 11898-1 sub-layer model explicitly, mapping each sub-layer to a dedicated module with a well-defined interface. The FCE becomes a standalone entity that receives error and success events from the MAC and can be exercised in isolation - `can_fce_tb` drives all counter-update rules directly without any frame-level stimulus. The explicit sub-layer boundaries also give the design direct traceability from module to standard: requirements assigned to a given layer (Section 6.1) point unambiguously to the corresponding module as the responsible implementation unit. This decomposition introduces inter-module interfaces and pipeline latency, but confines each concern to the sub-layer where it belongs.

6.2.2 Combined vs. Separated TX/RX FSMs

The initial design for the new MAC used **separate TX and RX FSMs**: `can_mac_fsm_tx` (~700 lines, 21 states) wrapped inside `can_mac_tx`, and `can_mac_fsm_rx` (~640 lines, 19 states) wrapped inside `can_mac_rx`. Each wrapper instantiated its own `can_mac_bs` and `can_mac_crc`. The two FSMs shared no state. The only coupling was a `transmitting_i` flag passed from TX to RX, and a dominant-wins OR-merge of their respective PCS outputs at the `can_mac` wrapper level.

The argument for the split was grounded in the verification plan structure. The AI-assisted extraction described in Section 3.3 had classified each requirement by side (TX, RX, both), layer, and frame format. Mapping that classification onto separate entities looked elegant: TX-side requirements would be verified by exercising `can_mac_tx` in isolation, RX-side requirements by exercising `can_mac_rx` in isolation, with no cross-path stimulus needed. The separation also appeared to offer independence - a bug in one path could not corrupt the other's state.

In practice the split created more problems than it solved. Frame structure - the field ordering, the bit stuffing rules, and the CRC polynomials - is identical regardless of which node is driving. Both roles traverse the same sequence of states. `is_transmitter` changes only a few lines of behavior within each state. In a split design all 19 states must therefore be present in both entities, and a fix to any shared

behavior - FD CRC delimiter handling, error-flag polarity, bit count initialization - must be replicated in both FSMs with no compiler enforcement that the copies stay in sync. The doubled submodule footprint similarly doubled investigation surface: every “is the bit stuffer handling fixed stuffing correctly?” question had to be answered for two independent instances. Two further structural costs compound the duplication. A transmitter must monitor the bus for bit errors and arbitration loss - nearly the same observation logic the receiver uses - so the TX FSM would have duplicated the RX FSM’s core bus-observation loop regardless of how the rest of the logic was split. And both FSMs can raise errors to the FCE simultaneously, requiring the FCE interface to arbitrate between duplicate error and success signals to avoid double-counting counter updates.

A further cost is debugging complexity. Single-bit-time bugs require a single-bit-time view of the frame position - but with two parallel FSMs, tracing any discrepancy requires correlating TX FSM state, TX bit count, RX FSM state, RX bit count, bit stuffer state on both sides, and CRC state on both sides simultaneously. Two independent state vectors that re-derive the same frame position independently double the investigation surface for every bug, with no compiler enforcement that the two derivations stay in sync.

The deeper lesson concerns the relationship between verification plan structure and RTL structure. Verification plan dimensions - TX/RX side, layer, frame format - are inputs to **testbench architecture**, not to RTL architecture. They describe what must be tested and what stimulus configuration is needed to test it. RTL architecture should follow the structure of the protocol itself.

The **final design** uses a single unified `can_mac_fsm` entity: one main FSM process (`p_fsm`), one `t_fsm_state` enum (19 states), one shared `can_mac_bs`, one shared `can_mac_crc`, and one `is_transmitter` mode flag latched at Start-of-Frame. TX-only requirements are verified with `is_transmitter = true` stimulus, RX-only with `is_transmitter = false` - the verification plan dimensions map cleanly onto testbench configurations rather than onto separate RTL entities.

6.2.3 Per-Field vs. Per-Phase FSM Granularity

The existing controller uses coarse-grained states that each cover multiple protocol fields: `s_arbitration` covers all ID bits, SRR, IDE, and RTR. `s_control` covers the reserved bit and DLC. `s_data` covers all data bytes. Within each state, a bit counter and conditional logic dispatch the correct action for each bit position. The FSM has 18 states.

The new FSM largely shares this structure. `s_arbitration` and `s_data` retain counter-driven logic for the same reason: both cover multi-bit fields with no per-bit semantic distinction. The meaningful change is in how format-specific single-bit fields are handled. Rather than folding FDF, RES, BRS, ESI, and SBC into a shared control state dispatched by counter, each gets a dedicated state. The same split applies to delimiter fields: `s_crc_delimiter` and `s_ack_delimiter` are separated from `s_crc` and `s_ack`. Format-dependent transitions become state graph edges - `s_fdf_r1_r0` transitions to `s_dlc` for Classic frames or to `s_res_r0` then `s_brs` for FD frames - rather than counter conditionals inside a shared state. The error frame sequence is simplified from five states to two (`s_error_flag`, `s_error_delimiter`), with fault confinement logic moved to the standalone `can_fce` entity. The result is 19 states - one more than the prior implementation - with counter logic confined to the fields that genuinely require it.

This structure aligns with the verification plan: FD-specific fields map directly to FSM states, and each state can be traced to its corresponding requirements. It also simplifies testbench assertions, since checks can reference named states rather than counter ranges.

6.2.4 Internal LLC Frame Format

The user-facing LLC frame (shown in Figure 11) places all control flags at the end of the frame: FDF, BRS, and ESI occupy byte 69, and IDE and RTR occupy byte 70 - after up to 64 bytes of payload. A serializer that consumed the LLC frame in field order would need to buffer the entire 71-byte frame before it could begin transmitting, because IDE determines how many ID bits to drive (11 or 29), FDF determines which CRC polynomial and stuffing mode to use, and BRS determines whether to signal the PCS to switch bit rate at the BRS boundary. This buffering requirement conflicts with the streaming architecture.

The design avoids this by defining a separate internal format for the MAC-facing stream, shown in Figure 9. All frame metadata is packed into two leading config bytes, followed by the ID and data bytes. With this layout, `can_mac_ser` extracts all frame metadata after receiving just two bytes and can begin streaming ID bits from the third byte onward. No frame buffering is needed: the MAC FSM receives each metadata field before it is required in the frame field sequence.

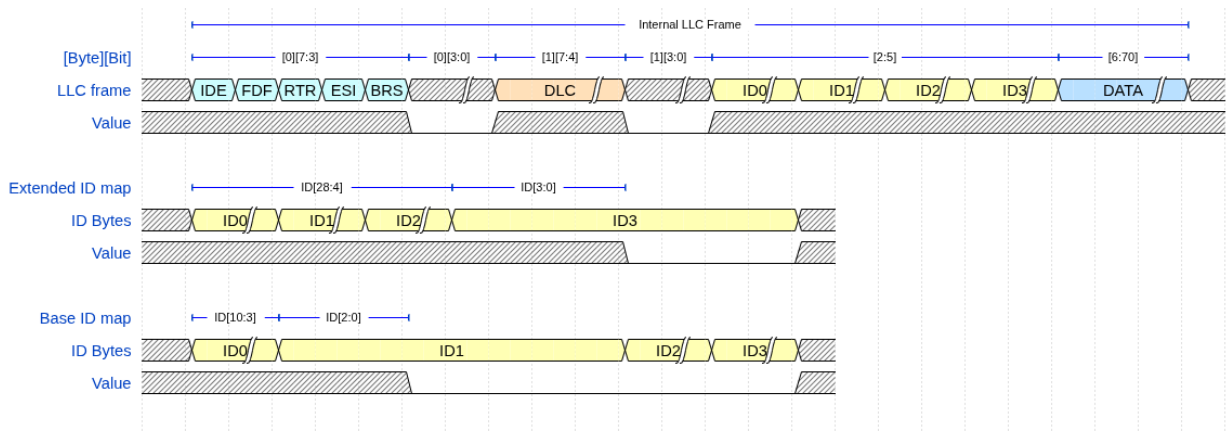


Figure 9: Internal LLC frame format at the `can_mac_ser` input, with identifier byte mapping for base and extended IDs.

6.3 System Overview

Figure 10 shows the complete module decomposition. The primary data path runs from `can_llc` through `can_mac` to `can_pcs`: the LLC receives frames from the host over an Avalon-ST interface and streams them byte-by-byte to the MAC serializer. The MAC FSM drives the serialized bit stream to the PCS, which applies bit timing and produces the sample-point and SSP strobes that the MAC uses to read and write the bus. `can_fce` sits outside the primary data path, receiving error and success events from the MAC and feeding node-state signals (error active, bus off) back to both the MAC and PCS. The PCS additionally pulses `can_fce` with idle conditions to drive bus-off recovery. A centralized types package (`can_types_pkg`) defines all protocol constants, interface records, and reset values shared across modules.

6.4 LLC Frame Format

All six bus frame types (CB, CE, FB, FE data frames, and remote frames for CB and CE) are represented at the host-LLC interface using the 71-byte LLC frame format shown in Figure 11. The LLC frame maps all in-scope variants into a fixed-width structure compatible with the Avalon-ST streaming interface, distinguishing remote frames via the FTYP bit in byte 70.

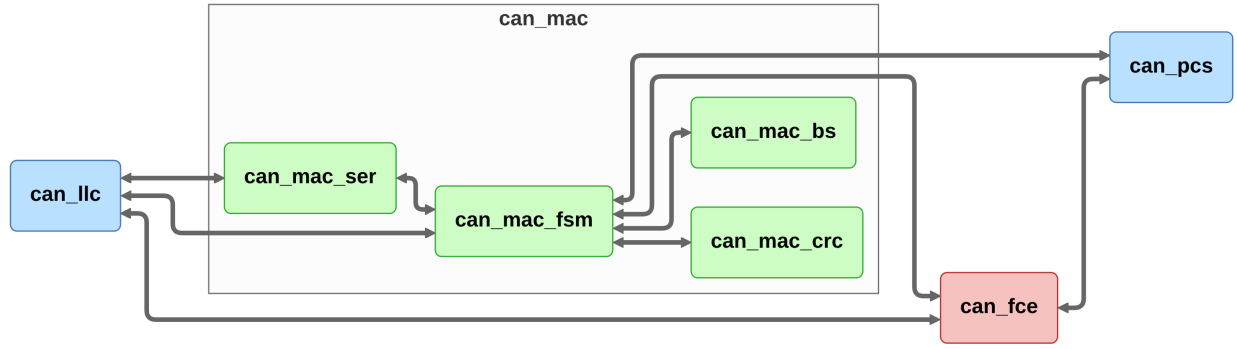


Figure 10: Implementation module decomposition showing the five entities and their inter-module connections.

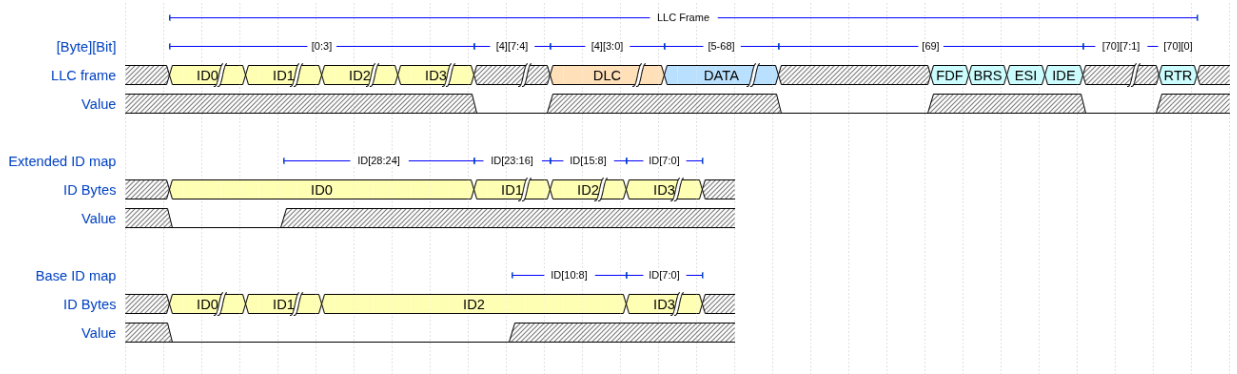


Figure 11: LLC frame format (71 bytes) at the host-LLC interface, with identifier byte mapping for base and extended IDs.

6.5 LLC Sub-layer

`can_llc` provides the Avalon-ST host interface and owns two protocol responsibilities above the MAC layer: retransmission management and acceptance filtering.

On the TX path, `can_llc` buffers one LLC frame from the host, streams it byte-by-byte to `can_mac_ser`, and monitors the transfer status returned by the MAC. On disturbance or lost arbitration, it retries transmission before reporting failure to the host [6, Sec. 6.4.5] and 6.5. The MAC reports each outcome (transmitted, aborted, lost arbitration, disturbed). `can_llc` owns the retry counter and policy - the MAC is stateless with respect to retry.

On the RX path, `can_llc` receives completed LLC frames from `can_mac_fsm` over the Avalon-ST interface, applies acceptance filtering against the configured ID mask, and forwards accepted frames to the host [6, Sec. 6.4.5]. Frames that do not pass the filter are silently dropped.

The interface contracts `can_llc` must satisfy are captured in REQ-001 through REQ-005 and REQ-033. The module is not yet implemented.

6.6 MAC Sub-layer

The MAC sub-layer is the core of the protocol logic, responsible for bit serialization, CRC generation, bit stuffing, and frame-level error detection. It is implemented as a single unified `can_mac_fsm` entity, supported by three internal submodules (`can_mac_ser`, `can_mac_bs`, `can_mac_crc`) and wrapped by `can_mac`, which is a structural entity exposing the LLC and PCS interfaces. `can_mac`, `can_fce`, and

`can_pcs` are then combined in the `can_mac_pcs_fce` wrapper. It coordinates closely with the FCE (Section 6.7) for error counter management and node-state transitions (error active/error passive/bus off), and with the PCS (Section 6.8) for sample-point-driven bit output.

6.6.1 `can_mac_fsm`

`can_mac_fsm` is a 19-state per-field FSM that handles both frame transmission and frame reception. An `is_transmitter` flag latched at SOF partitions per-state logic into TX and RX branches without duplicating the state graph. In TX mode, the FSM drives bits through the PCS at each bit-boundary strobe, monitors the bus echo for bit errors and arbitration loss, and feeds the bit stuffer and CRC engine. In RX mode, it observes sampled bus bits, performs destuffing, accumulates the CRC, validates form fields, drives the ACK slot, and streams the completed frame to the LLC during intermission. Errors in either mode branch to a two-state error-frame sequence (`s_error_flag`, `s_error_delimiter`). Detailed implementation is described in Section 7.2.

6.6.2 `can_mac_ser`

`can_mac_ser` converts the LLC byte stream into a serial polarity bit stream for the MAC FSM. It manages the two-byte configuration handshake (see Section 6.2.4), extracts LLC metadata (IDE, FDF, DLC, FTYP, BRS, ESI) from the config bytes, and serializes ID and data bits one per FSM ready pulse. Detailed implementation is described in Section 7.3.

6.6.3 `can_mac_bs`

`can_mac_bs` implements both dynamic and fixed bit stuffing for CAN Classic and CAN FD frames [6, Sec. 10.6], used for both TX stuffing and RX destuffing via the FSM's `is_transmitter` mode. In dynamic mode, it inserts an inverse-polarity bit after five consecutive identical bits. In fixed mode (FD CRC region), it inserts one bit on entry then one every four bits, and maintains a Gray-coded stuff-bit count with parity for the SBC field. Detailed implementation is described in Section 7.4.

6.6.4 `can_mac_crc`

`can_mac_crc` runs three parallel `gen_crc` instances - CRC-15, CRC-17, and CRC-21 - on two independent data feeds: `data_cc` (destuffed, for Classic frames) and `data_fd` (includes dynamic stuff bits in the arbitration region, for FD frames). The active engine is selected by `crc_poly_select` and its result is left-aligned to a common 21-bit output. Detailed implementation is described in Section 7.5.

6.7 FCE Sub-layer

`can_fce` implements the error FSM and counter management specified in ISO 11898-1 [6, Secs. 8.1.3–8.1.4]. It maintains TEC and REC and transitions between three states: `s_error_active` (initial), `s_error_passive` (TEC or REC > 127), and `s_bus_off` (TEC > 255). Counter increment and decrement rules follow [6, Sec. 8.1.4.2]. Bus-off recovery counts 128 `pcs_i.idle_condition` pulses from the PCS (11 consecutive recessive bits each). `llc_i.normal_mode` is a supervisory reset per ISO 11898-1 that the LLC may assert from any node state to immediately reset TEC, REC, and the FCE FSM to their initial values. Detailed implementation is described in Section 7.6.

6.8 PCS Sub-layer

`can_pcs` handles bit timing, hard synchronization and resynchronization, and Transmitter Delay Compensation (TDC) for both TX and RX paths [6, Secs. 7.2–7.3]. It is a cyclic bit-timing engine: a `t_segment` register cycles through `s_sync_seg`, `s_prop_seg`, `s_phase_seg1`, and `s_phase_seg2` each bit time. The SP strobe fires at the end of `s_phase_seg1`. Hard synchronization restarts the cycle on a dominant edge in `s_sync_seg`. Resynchronization adjusts `s_phase_seg1` or `s_phase_seg2` by up to SJW. At the BRS sample point the PCS switches to the independently configured data-phase segment lengths. TDC measures the TX-to-RX echo delay at the first data-phase bit and positions the SSP accordingly. When bus-off is asserted, the PCS counts consecutive recessive bits and pulses `fce_o.idle_condition` every 11 bits for FCE recovery. Detailed implementation is described in Section 7.7.

The decomposition described above yields six implemented entities - `can_mac_fsm`, `can_mac_ser`, `can_mac_bs`, `can_mac_crc`, `can_fce`, and `can_pcs` - plus `can_mac` and `can_mac_pcs_fce` as structural wrappers and `can_llc` as the one module not yet implemented. Section 7 covers the implementation of each entity in turn, ordered MAC-first.

7 Implementation

Section 6 concluded that a unified `can_mac_fsm` - one FSM, one bit stuffer, one CRC engine - was the right architecture once the split-path approach was attempted and found unworkable. This section shows what that architecture looks like in practice. Two threads run through the module descriptions that follow. The first is the set of implementation decisions that were not derivable from the requirements table alone but were forced by protocol structure during implementation: the bit stuffer’s handling of a pending dynamic stuff bit at the dynamic-to-fixed mode boundary, the CRC engine’s combinatorial output mux and why a registered stage would have broken the frame timing, and the PCS synchronization rules that the prior implementation violated. The second is the places where the unified FSM design decision pays off - where a protocol rule that applies equally to transmitter and receiver is expressed once in shared state rather than twice in parallel FSMs. Subsections are ordered MAC-first: the FSM and its internal submodules (`can_mac_ser`, `can_mac_bs`, `can_mac_crc`) are described before the supporting `can_fce` and `can_pcs` layers, so that the FSM’s interface contracts are established before the modules that satisfy them. `can_llc` is not yet implemented. Its interface contracts are captured in the verification plan (REQ-001 through REQ-005, REQ-033).

7.1 Interface Conventions

Everllence’s `std_logic`-only port constraint does not preclude the use of VHDL record types on entity ports - records of `std_logic` and `std_logic_vector` fields satisfy the constraint. The design uses typed record interfaces (e.g., `t_can_mac_pcs_if_m2s`, `t_can_mac_fsm_bs_if_s2m`) to bundle related signals into a single port. Each record type has a corresponding reset constant (e.g., `c_can_mac_pcs_if_m2s_reset`), ensuring that every module can be reset to a known state without manually enumerating each field.

The alternative - flat port lists with individual `std_logic` signals - was used in the existing controller, where the FSM entity has 20 individual ports for its various submodule interfaces. This approach becomes unwieldy as inter-module signal counts grow: bundling related signals into records reduces each entity’s port list to a handful of typed ports, keeping the interface manageable as the design scales.

The naming convention follows the data-flow direction: `m2s/s2m` (master-to-slave/slave-to-master) for con-

trol interfaces, and s2d/d2s (source-to-destination/destination-to-source) for Avalon-ST data-transfer interfaces. This convention is consistent with Everllence's existing interface naming and makes the direction of data flow explicit at every port.

7.2 can_mac_fsm

The can_mac sub-layer is built around a single unified FSM entity (can_mac_fsm, ~1100 lines). The full per-signal interface is shown in Figure 12.

7.2.1 FSM Structure and Mode Flag

can_mac_fsm contains two synchronous processes (p_fsm and p_stream_to_LLC) and one t_fsm_state enum covering 19 states. An is_transmitter boolean signal is latched to true when the FSM drives the SOF dominant bit at the start of a new frame transmission and cleared at arbitration loss or at the end of the EOF field. Once latched, is_transmitter remains stable for the rest of the frame, partitioning per-state logic into a TX branch (drive_bit strobe: determines and drives pcs_o.tx_data) and an RX branch (sample_point strobe: advances state and captures received bits). The state transitions at the sample point are shared between TX and RX in most states. Only s_ack, s_ack_delimiter, and s_eof carry role-specific logic on the sample-point path (ACK success latching, receiver dominant assertion, and frame-completion signaling respectively).

The per-field state granularity introduced in Section 6.2.3 is preserved in the unified FSM. The arbitration region uses two states: SOF is driven in s_bus_idle, and the ID bits, RTR/SRR/RRS, and IDE share s_arbitration with bit_count indexing into the 32-bit ID field. Each post-arbitration protocol field (FDF, RES, BRS, ESI, DLC, Data, SBC, CRC, CRC Delimiter, ACK, ACK Delimiter, EOF) has a dedicated state, making field boundaries explicit in the state encoding. The complete FSM is shown in Figure 13.

7.2.2 TX Mode: Frame Transmission

When is_transmitter = true, the FSM drives bits through pcs_o.tx_data at the bit-boundary strobe (drive_bit, generated internally by registering pcs_i.sample_point twice). On each sample-point strobe the FSM executes the following four-step sequence per frame-field state:

1. **Monitor the bus.** The sampled bus polarity (pcs_i.rx_data) is compared against the previously transmitted bit (held in a 32-bit polarity history shift register, transmitted_bits_shift_reg) to detect bit errors, ACK, and arbitration loss. In the CAN FD data phase, the SSP strobe fires once per bit time. The MAC compares pcs_i.rx_data against transmitted_bits_shift_reg(tdc_delay) - where tdc_delay is supplied alongside the SSP strobe by the PCS - to check the bit that was transmitted tdc_delay bit times earlier [6, Sec. 7.3.4]. A detected error triggers a transition to the error-frame sequence with the appropriate flag type based on FCE fault confinement status [6, Secs. 8.1.3–8.1.4]. Arbitration loss causes is_transmitter to flip to false in-place, and the s_arbitration state then continues as an RX observer for the remaining bits.
2. **Determine the next bit.** If the bit stuffer has a pending stuff bit (bs_i.valid), that takes priority. Otherwise, the polarity is determined by the current state: form bits (SOF, IDE, FDF, reserved, delimiters, EOF) have fixed polarities, while ID, DLC, data, SBC, and CRC bits are sourced from the serializer, metadata, stuff-bit count, or CRC register respectively.
3. **Feed the CRC engine and bit stuffer.** The feed source is less obvious than it appears. The post-case feed is pcs_i.rx_data throughout s_arbitration - for both transmitter and receiver. This

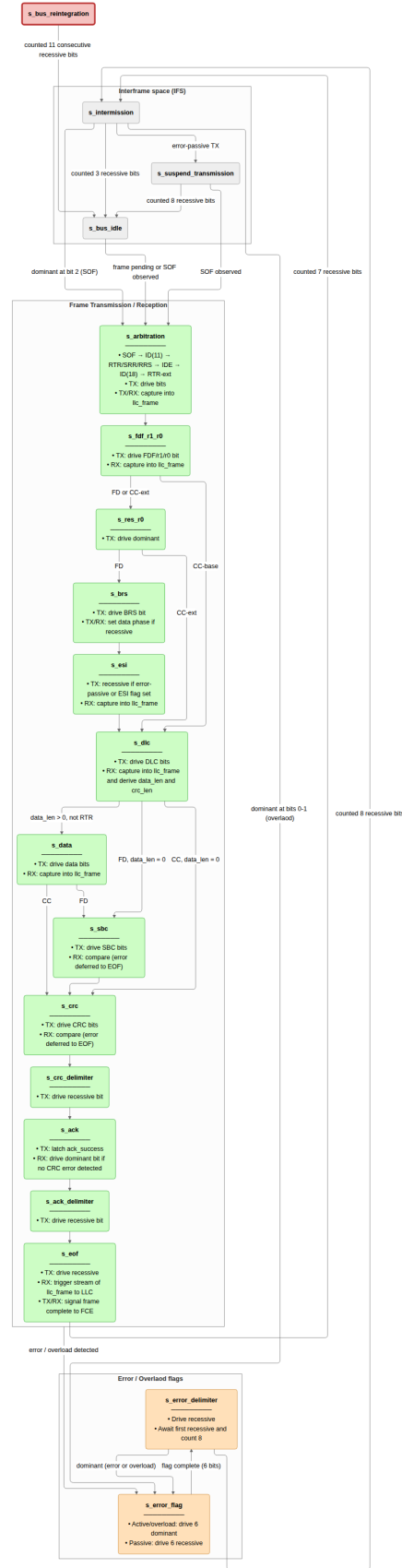


Figure 13: `can_mac_fsm` (19 states) controlling TX and RX for all in-scope frame formats. Arbitration loss clears `is_transmitter` in place without a state transition.

is necessary for arbitration-loss correctness: if the transmitter loses arbitration and continues as a receiver, its CRC accumulator must hold exactly the value a pure receiver would have built, so both roles must feed from the actual bus value. From `s_fdf_r1_r0` onward the transmitter switches to `transmitted_bits_shift_reg(0)`, which is safe because the transmitter owns the bus unconditionally after arbitration. This also avoids any dependency on `pcs_i.rx_data` echo latency once the data phase and TDC are active. The FSM asserts `bs_o.fixed_bit_stuffing_en` when entering the SBC field of FD frames to switch the bit stuffer from dynamic to fixed mode.

4. **Present the bit at the PCS interface.** The resolved polarity is written to `pcs_o.tx_data` and becomes visible at `tx_o` when the bit-boundary latch fires. `pcs_o.next_bit_is_brs` is asserted one SP before the BRS bit, allowing the PCS to switch to data-phase timing at the BRS SP if BRS is sampled recessive. `pcs_o.next_bit_is_res` is asserted one SP before the FD reserved bit to arm TDC measurement at the subsequent bit boundary [6, Sec. 7.3.4].

7.2.3 RX Mode: Frame Reception

When `is_transmitter = false`, the FSM observes `pcs_i.rx_data` at each sample-point strobe and stores received bits directly into an internal `llc_frame` byte array. No separate deserializer is needed. The bit stuffer is driven from `pcs_i.rx_data` to perform destuffing, and the CRC engine accumulates the received bit stream in parallel. The FSM validates the SBC field (FD frames), compares the received CRC against the locally accumulated result, and checks form bits (reserved bits, CRC delimiter, ACK delimiter, EOF) for required polarities. A mismatch in any of these fields triggers a transition to the error-frame sequence. During the ACK slot the FSM drives `pcs_o.tx_data = c_dominant` for one bit (`bit_count = 0`) regardless of frame format. The FD ACK slot spans two bits but the receiver asserts dominant only during the first. The bus is released after the ACK delimiter [6, Sec. 8.1.4.2.b].

After the EOF field the FSM transitions through `s_intermission`. A dedicated second process, `p_stream_to_LLC`, transfers the completed `llc_frame` array byte-by-byte to the LLC RX sink over the Avalon-ST interface. `p_fsm` triggers the transfer by asserting `llc_stream_start` and separately signals successful reception to the FCE. This design eliminates the need for a separate deserializer entity on the RX path - the frame buffer is populated and streamed entirely within `can_mac_fsm`.

7.2.4 Error-Frame States

The FSM uses two explicit error-frame states. `s_error_flag` drives the 6-bit flag and `s_error_delimiter` counts the 8-bit recessive delimiter. The delimiter state manages an internal phase flag (`delim_found_first_recessive`) that separates two distinct sub-phases: first awaiting the bus to go recessive (other nodes may still be driving their own flags), then counting the remaining recessive bits. A dominant during the delimiter restarts the error-frame sequence - either as a new error or as an overload condition on the last delimiter bit [6, Sec. 8.1.4.2.f]. Both transmitter and receiver errors enter this same two-state sequence: the flag polarity is not `fce_i.error_active`, determined by the FCE node state rather than the TX/RX role, and `s_error_delimiter` is entirely role-independent.

The three submodules the FSM depends on - the serializer for the TX bit stream, the bit stuffer for stuff-bit insertion and SBC generation, and the CRC engine for parallel polynomial accumulation - are described in Section 7.3, Section 7.4, and Section 7.5.

7.3 can_mac_ser

can_mac_ser converts the LLC byte stream into a serial polarity bit stream for the MAC FSM. Its four-state FSM manages the two-byte configuration handshake, byte fetching, and bit-by-bit serialization. The serializer extracts LLC metadata (IDE, FDF, DLC, FTYP, BRS, ESI) from the two config bytes and registers it in `t_llc_metadata`, which remains stable for the entire frame. The internal frame format that makes this possible is described in Section 6.2.4. The serializer forwards `transfer_status` from the FSM back to the LLC, returning to `s_load_config_byte_0` on any non-ongoing status so that errors and aborts terminate serialization immediately.

The 32-bit ID field in the internal format is right-aligned: an extended identifier (29 bits) uses bits [28:0], leaving 3 unused bits. A base identifier (11 bits) uses bits [10:0], leaving 21 unused bits. The serializer tracks this with two counters initialized in `s_load_config_byte_1` from the `ide` flag: `id_bits_remaining` counts real ID bits still to be presented, and `padding_bits_remaining` counts leading zeros to be skipped. In `s_shift_out_bits`, padding bits are advanced without asserting `valid`, so the MAC FSM never observes them. This means the FSM always receives exactly 11 or 29 consecutive valid ID bits regardless of frame format, with no format-specific logic required downstream.

Bit serialization uses a single-byte shift register. On entry to `s_shift_out_bits`, `llc_frame_buffer` holds the current byte with the MSB pre-loaded into `tx_mac_fsm_o.data`. The serializer holds `valid` high while a real bit is waiting. The FSM acknowledges by asserting `ready` for one cycle. On each accepted transfer the buffer is shifted left by one and the new MSB is presented as the next bit. When the final bit of the byte is consumed (`count = c_byte_width - 1`), `valid` is deasserted and the FSM returns to `s_load_llc_frame_byte` to fetch the next byte. The serializer deasserts its own `ready` toward the LLC during `s_shift_out_bits`, stalling the LLC from presenting the next byte until the current one is fully drained. The four-state serializer FSM is shown in Figure 14.

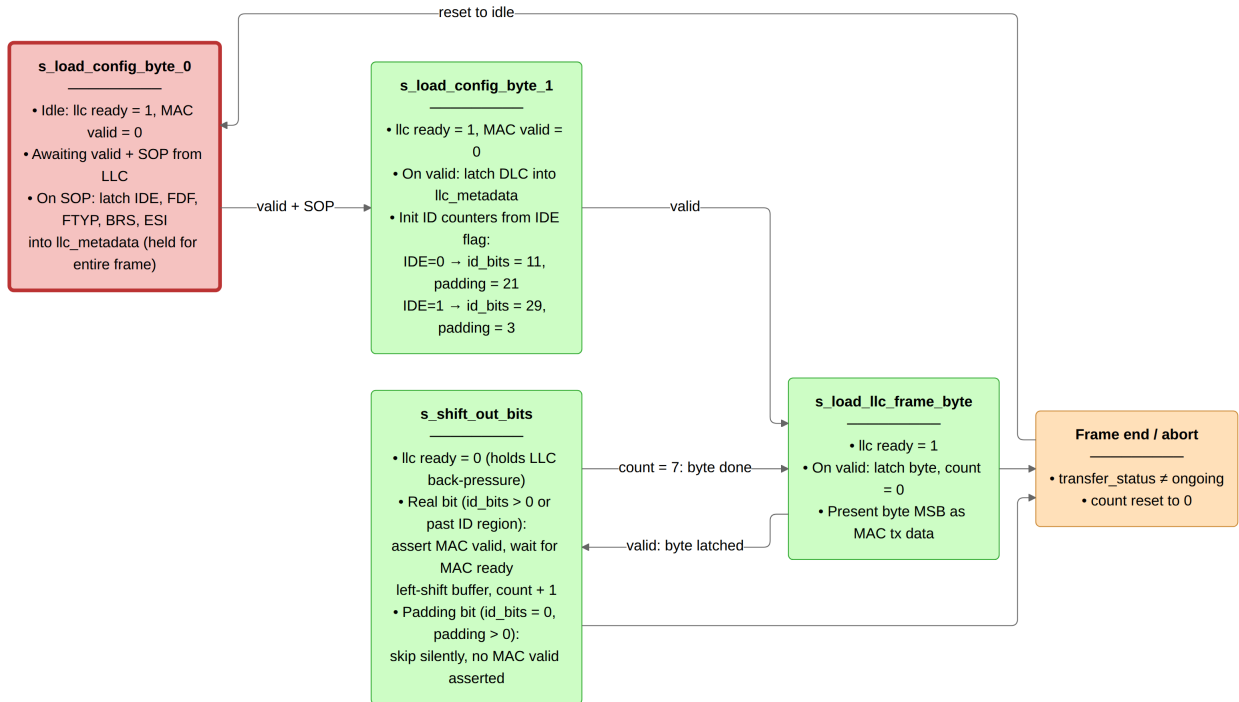


Figure 14: can_mac_ser FSM (four states) serializing the internal LLC frame to the MAC bit stream. Unused padding bits in the 32-bit ID field are skipped silently.

The bit stream it produces feeds the bit stuffer, which may insert additional stuff bits before the FSM drives each bit to the PCS interface.

7.4 can_mac_bs

can_mac_bs implements both dynamic and fixed bit stuffing for CAN Classic and CAN FD frames [6, Sec. 10.6]. The single entity is instantiated once inside can_mac_fsm and serves both TX stuffing and RX destuffing via the same logic - the FSM drives the same bs_i interface regardless of role, and the stuffer's output is either inserted into the TX bit stream or used by the FSM to discard a received destuff bit. A split-path design would require two independent stuffer instances with identical logic.

In **dynamic mode** (fixed_bit_stuffing_en = '0'), the stuffer counts consecutive bits of identical polarity and emits an inverse-polarity stuff bit after every five (REQ-019). The Gray-coded stuff-bit count stuff_count is incremented on each dynamic stuff bit and encoded with parity into the stuff_bit_count output, which the FSM reads when transmitting the SBC field (REQ-017).

In **fixed mode** (fixed_bit_stuffing_en = '1'), used for the FD CRC region, one fixed stuff bit (FSB) is emitted immediately on the rising edge of fixed_bit_stuffing_en, then one FSB every four real bits (REQ-019). The FSB polarity is always the inverse of the preceding bit, so a receiver can detect a form error if the FSB matches its predecessor.

The transition from dynamic to fixed stuffing requires special handling when a dynamic stuff bit is already pending at the rising edge of fixed_bit_stuffing_en. Suppressing the pending dynamic SB would cause a TX/RX divergence: the transmitter and receiver would derive different stuff_count values from the same bit stream, causing SBC mismatch. The implementation instead promotes the pending dynamic SB to the initial FSB - the two coincide and both requirements are satisfied simultaneously (see Figure 15, REQ-017, REQ-019). On the falling edge of fixed_bit_stuffing_en, any pending FSB is cancelled immediately. The MAC FSM exits fixed stuffing at the last CRC bit without providing a slot to drain a still-pending FSB.

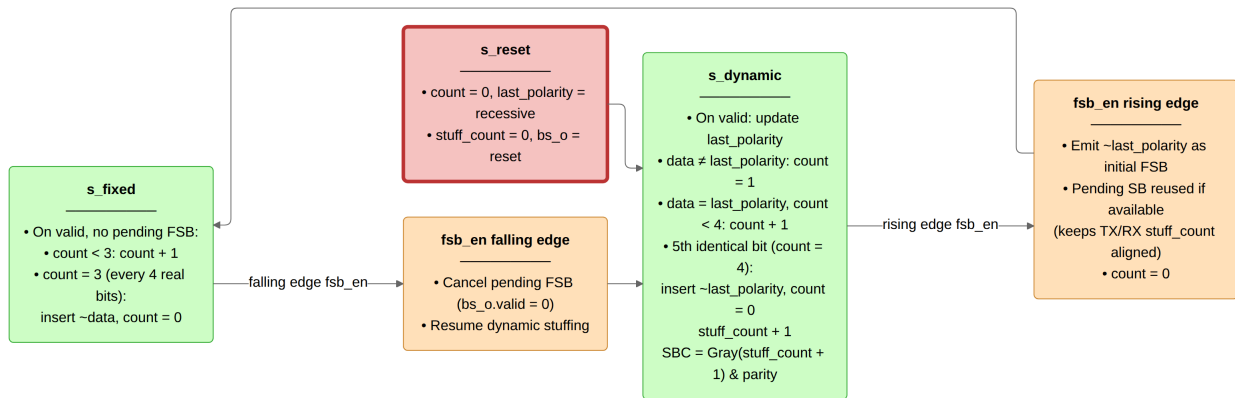


Figure 15: can_mac_bs operating in dynamic and fixed stuffing modes. A pending dynamic stuff bit at the rising edge of fsb_en is promoted to the initial FSB rather than suppressed.

Its stuff_bit_count output is the SBC value the FSM reads when transmitting the SBC field. The CRC engine, which consumes the pre-stuff and post-stuff streams simultaneously on two independent feeds, is described next.

7.5 can_mac_crc

CAN Classic computes its CRC over the raw bit stream excluding stuff bits, while CAN FD includes dynamic stuff bits in the arbitration region - a difference that exists because the FD CRC must protect the stuff-bit count as well as the data. A single data feed to the CRC engine is therefore insufficient: CC and FD frames require different input streams. The design exposes two feeds on the CRC interface (`data_cc` and `data_fd`) so the FSM can drive both simultaneously and the CRC module requires no protocol knowledge about which stream to select.

`can_mac_crc` provides CRC generation and checking for both CAN Classic and CAN FD frames. CAN Classic frames use CRC-15, while CAN FD frames use CRC-17 (data payloads up to 16 bytes) or CRC-21 (data payloads above 16 bytes) [6, Sec. 10.4.2.6]. The single entity is instantiated once inside `can_mac_fsm`, serving both TX (generation) and RX (checking). A split-path design would require two separate sets of three `gen_crc` instances - six engines instead of three. The FSM sets `crc_poly_select` from the DLC field in `llc_metadata` before the first frame bit is driven: because the internal LLC frame format (Section 6.2.4) delivers DLC in config byte 1, the polynomial is known upfront and requires no mid-frame switching.

Three parallel `gen_crc` instances run continuously on separate data feeds: `data_cc` drives CRC-15 via `valid_cc`, while `data_fd` drives both CRC-17 and CRC-21 via `valid_fd`. This dual-feed architecture is necessary because CC and FD frames compute CRC over different bit streams - CC excludes stuff bits while FD includes them in the arbitration region - and the RX path does not know which CRC engine to use until after the frame type has been determined. The output multiplexer selects the active engine's result based on `crc_poly_select` and left-aligns it to the common 21-bit output width by zero-extending the shorter results at the least significant bit: CRC-15 occupies bits [20:6], CRC-17 occupies bits [20:4], and CRC-21 occupies the full width.

The output mux (`p_crc_mux`) is implemented combinatorially rather than as a registered stage. Each `gen_crc` instance registers its accumulator on the rising edge. A registered mux would add one cycle of latency, causing the FSM to read a stale digest on the cycle it drives the first CRC bit. The combinatorial mux ensures `crc_o.crc` reflects the fully accumulated result on the same cycle that the last data bit is registered, so the FSM can begin driving CRC bits immediately without an explicit wait state (Figure 16).

With the MAC submodules established, the two remaining modules - the Fault Confinement Entity and the Physical Coding Sublayer - are described in Section 7.6 and Section 7.7.

7.6 can_fce

`can_fce` implements the error FSM and counter management specified in ISO 11898-1 [6, Secs. 8.1.3–8.1.4]. It maintains TEC (Transmitter Error Counter) and REC (Receiver Error Counter) and transitions between three states: `s_error_active` (normal operation), `s_error_passive` (TEC or REC > 127), and `s_bus_off` (TEC > 255), as shown in Figure 17.

Counter updates follow the rules in [6, Sec. 8.1.4.2]: TEC increments by 8 on TX errors, with `mac_i.passive_tx_ack_error_exempt_1` suppressing the increment for the passive ACK error exemption (ISO 8.1.4.2.c, Exception 1). TEC decrements by 1 on successful TX. REC increments by 1 on RX errors during non-error-flag phases, by 8 on primary errors or error-flag-phase errors, and decrements by 1 or clamps to 127 on successful RX. Bus-off recovery requires counting 128 `pcs_i.idle_condition` pulses (11 consecutive recessive bits each) from the PCS, which resets both counters and returns the FSM to `s_error_active`. `llc_i.normal_mode` is a supervisory reset per ISO 11898-1 that forces the same transition from any node state immediately.

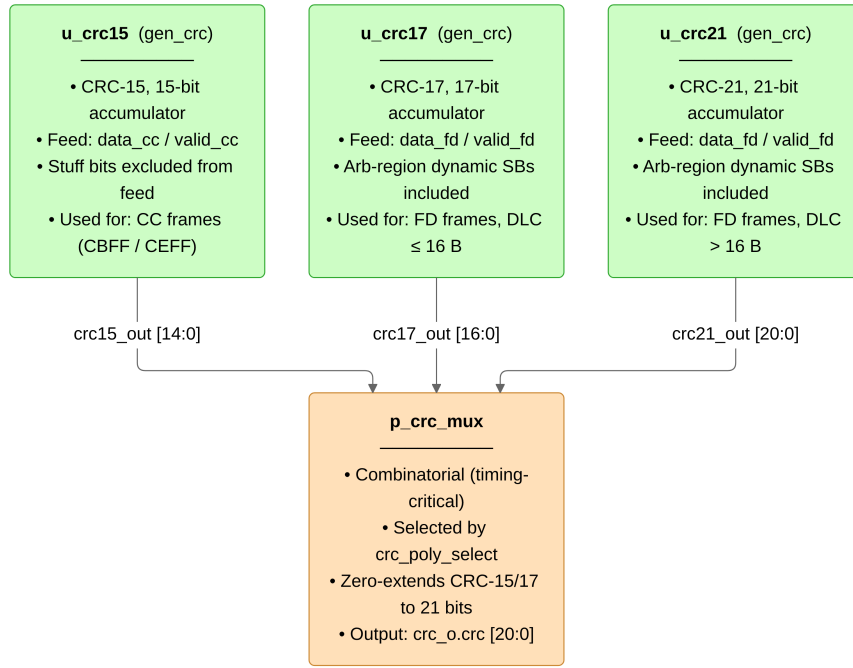


Figure 16: `can_mac_crc` dataflow with three parallel CRC engines. The output mux is combinatorial to avoid one cycle of read latency at the FSM.

The one counter rule that requires careful reading of the ISO prose is the passive ACK error exemption (ISO 8.1.4.2.c, Exception 1): an error passive node that transmits a frame and receives no dominant ACK bit shall not increment TEC, because the node's passive error flag is recessive and may itself prevent receivers from asserting the ACK slot. The FCE has no frame-level visibility - it receives event signals from the MAC, not raw bus bits - so the MAC must explicitly signal this case via `mac_i.passive_tx_ack_error_exempt_1`, asserted when the FSM detects an ACK error while `mac_o.error_active` is deasserted. Without this signal the FCE would treat an unacknowledged passive-node transmission identically to any other ACK error and escalate TEC unnecessarily.

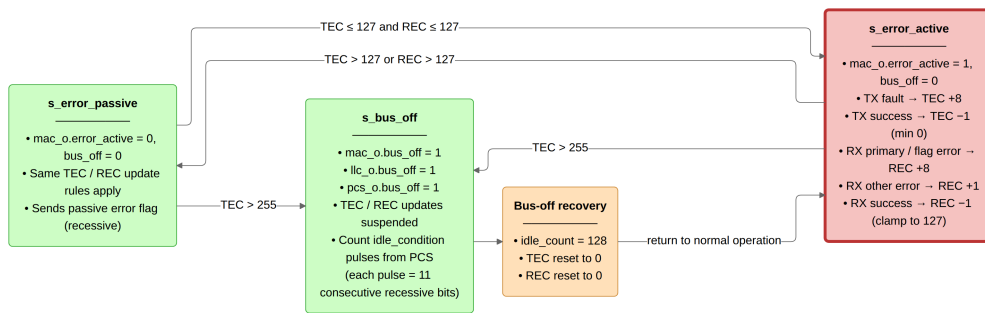


Figure 17: `can_fce` FSM governing the error active, error passive, and bus off node states per ISO 11898-1 sec. 8.1.4.4.

The PCS layer, which supplies the bit-level timing strobes that drive every FSM state transition, is described next.

7.7 can_pcs

`can_pcs` is a cyclic bit-timing engine: its internal `t_segment` register advances through `s_sync_seg` (1 TQ, fixed), `s_prop_seg`, `s_phase_seg1`, and `s_phase_seg2` on every TQ boundary. The SP

strobe and rx_data latch fire at the end of s_phase_seg1. The TX bit is driven at the end of s_phase_seg2. When fce_i.bus_off is asserted, the SP slot counts consecutive recessive bits and pulses fce_o.idle_condition every 11 bits for FCE bus-off recovery. The full timing operation is shown in Figure 18.

7.7.1 Resynchronization

The prior implementation missed three of the four ISO 7.3.5.1 synchronization rules: it had no sync-inhibit guard (rule a), no sampled-polarity check (rule b), and a Phase_Seg2 shortening path that skipped the mandatory 1-TQ Sync_Seg (rule d). None of these caused observable failures on the deployed CAN Classic bus, but all three are protocol obligations. can_pcs enforces all four.

Rule a - one synchronization per bit time. A sync_applied signal is set on any synchronization event (hard synchronization or resynchronization) and cleared at the next bit boundary (end of s_phase_seg2). The TQ-boundary edge-qualify predicate v_do_sync includes sync_applied = '0' as a precondition, preventing a second synchronization within the same bit time regardless of bus activity.

Rule b - sync only on a recessive-to-dominant transition. The edge-qualify predicate requires rx_bus_prev = c_recessive (the bus value latched at the preceding TQ boundary) together with rx_i = c_dominant, making synchronization conditional on an actual recessive-to-dominant edge. This prevents spurious synchronization on a dominant-to-recessive-to-dominant glitch within a dominant bit. A further guard, mac_i.transmitting = '0', disables synchronization entirely while the local node is driving the bus.

Rule c - hard synchronization on demand. Rather than triggering hard synchronization on the first dominant edge following reset (as the prior implementation did), can_pcs accepts a MAC-driven mac_i.do_hard_sync signal. When asserted, any qualifying edge triggers a full bit-time restart from s_prop_seg with the prescaler and segment counter cleared. This allows the MAC to switch synchronization mode at any point - including at the FDF-to-res transition required by ISO 7.3.5.1(c) - without resetting PCS timing state.

Rule d - Sync_Seg always traversed. s_sync_seg is an unconditional stop in the segment FSM: s_phase_seg2 always transitions to s_sync_seg at the bit boundary, and s_sync_seg always transitions to s_prop_seg after 1 TQ. No shortcut paths exist.

7.7.2 Dual Bit Rate Switching

can_pcs holds no frame-format knowledge. Rate switching is entirely MAC-driven through three dedicated control signals on the MAC-PCS interface.

mac_i.next_bit_is_brs is asserted one SP before the BRS bit. At the BRS SP, can_pcs reads rx_i (or mac_i.tx_data when transmitting) to determine BRS polarity. If recessive, it replaces active_prop_seg, active_phase_seg1, active_phase_seg2, and active_sjw with the data-phase generics (gc_data_prop_seg, gc_data_phase_seg1, gc_data_phase_seg2, gc_data_sjw) and sets data_phase_active. mac_i.next_bit_is_res is asserted when the next bit is the FD reserved bit. At the corresponding bit boundary, tdc_count_active is set to begin TDC measurement (described in Section 7.7.3). mac_i.data_phase_stop is asserted by the MAC at the CRC delimiter SP or on entry to the error-frame sequence. It restores nominal timing parameters and clears all TDC state.

This interface design keeps protocol knowledge in the MAC layer and timing knowledge in the PCS layer,

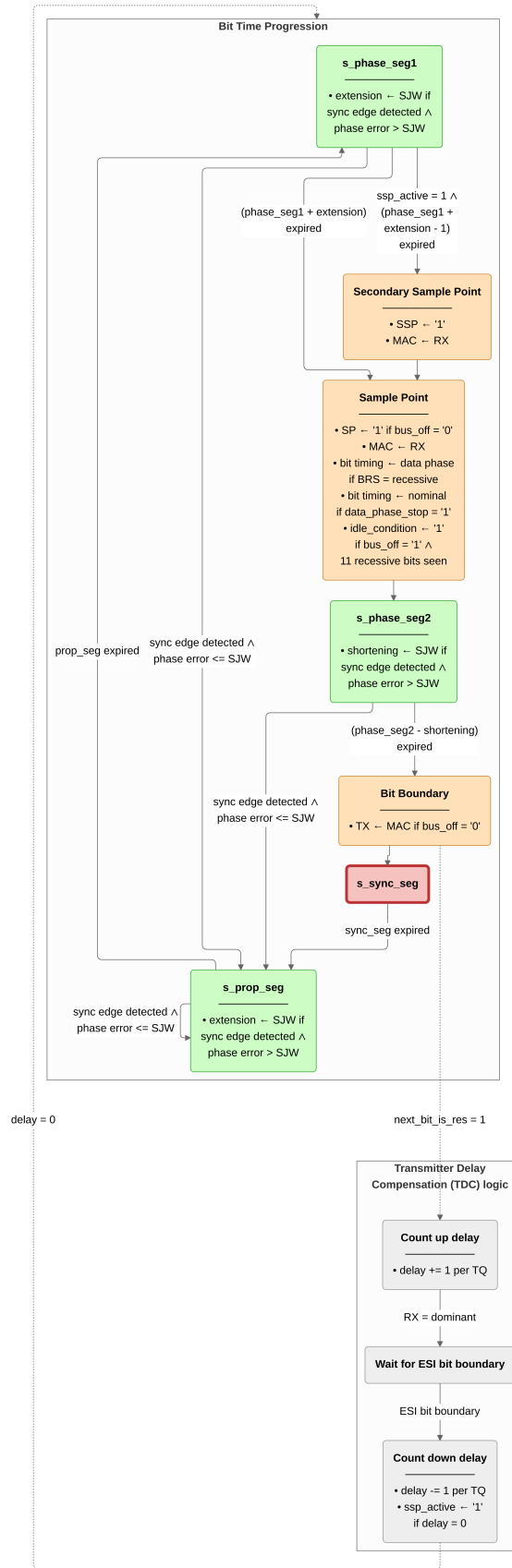


Figure 18: `can_pcs` bit-time FSM with concurrent resynchronization and TDC pipelines per ISO 11898-1 sec. 7.2-7.4.

following the ISO 11898-1 layered architecture. `can_pcs` does not inspect DLC or frame format - the only frame-level observation it makes is reading the BRS bit polarity when `mac_i.next_bit_is_brs` is set, making it independently testable against any timing configuration without frame-level stimulus.

7.7.3 Transmitter Delay Compensation

The motivation and principle of TDC are described in Section 4.3. The implementation in `can_pcs` is flag-based logic within the single `p_can_pcs` process, not a separate FSM. The relevant signals are `tdc_count_active`, `delay_count_tq`, `ssp_standoff_active`, `first_data_bit_boundary_seen`, `ssp_active`, `ssp_seen`, and `tdc_delay`.

TDC is armed at the bit boundary when `mac_i.next_bit_is_res = '1'`, which sets `tdc_count_active`. While set, `delay_count_tq` increments by one per recessive TQ at each TQ boundary. On the first dominant TQ - the TX-to-RX echo of the data-phase preamble - `tdc_count_active` is cleared, leaving `delay_count_tq` holding the measured round-trip delay in TQs.

At the first data-phase bit boundary, `first_data_bit_boundary_seen` is latched and `ssp_standoff_active` is asserted. `delay_count_tq` then counts down one per TQ from its measured value. In parallel, `tdc_delay` increments once at each subsequent bit boundary while `ssp_seen = '0'`, counting how many whole bit periods the standoff spans. When `delay_count_tq` reaches zero, `ssp_active` is set and `ssp_standoff_active` is cleared. With `ssp_active = '1'`, the SSP strobe fires at a fixed offset before the SP within `s_phase_seg1` every data-phase bit time, latching `rx_i` and pulsing `mac_o.secondary_sample_point`. On the first SSP fire, `ssp_seen` is latched and `mac_o.tdc_delay` is captured at the current `tdc_delay` value. The MAC reads the stable `mac_o.tdc_delay` to index into `transmitted_bits_shift_reg`, identifying the transmitted bit whose loopback echo is being observed at the SSP.

`mac_i.data_phase_stop` at the SP clears `ssp_active`, `ssp_seen`, `tdc_count_active`, `delay_count_tq`, `tdc_delay`, `data_phase_active`, and `first_data_bit_boundary_seen`, restoring nominal SP-based monitoring for the CRC delimiter and subsequent fields.

The six entities described above - `can_mac_fsm`, `can_mac_ser`, `can_mac_bs`, `can_mac_crc`, `can_fce`, and `can_pcs` - together with the structural wrappers `can_mac` and `can_mac_pcs_fce` constitute the implemented protocol engine. The two threads named at the start of this section were both borne out. Each module contained at least one implementation decision that was not visible in the requirements table and only became concrete under the full protocol constraints: the bit stuffer's mode-boundary promotion rule, the CRC engine's combinatorial mux, and the PCS synchronization guard are the clearest examples. And in each case, the unified-FSM architecture simplified the fix - a single correction to a shared submodule propagated to both TX and RX paths automatically. With the implementation complete, the remaining question is whether the 38 requirements in the verification plan are in fact satisfied by what was built - the subject of Section 8.

8 Verification and Results

The implementation described in Section 7 was exercised against the 38-requirement verification plan (Section 5) using five dedicated testbenches, each aligned to a module boundary established by the layered architecture. `can_mac_pcs_fce_tb` is the primary integration testbench, exercising `can_mac_fsm` together with `can_pcs` and `can_fce` in a two-node loopback configuration and covering 16 requirements spanning MAC frame encoding, PCS bus-off recovery, and FCE-driven error flag generation. The

four unit testbenches - `can_mac_crc_tb`, `can_mac_bs_tb`, `can_pcs_tb`, and `can_fce_tb` - target individual submodules with focused stimulus. Two additional testbenches, `can_mac_ser_tb` and `can_llc_mac_pcs_fce_tb`, exercise the serializer and the full MAC-PCS-FCE stack respectively but have no standalone requirement entries in the verification plan. [REQ-013](#) (CC CRC data feed excludes stuff bits) and [REQ-024](#) (overload frame conditions) are verified by code inspection against `can_mac_fsm.vhd` rather than simulation, as both reduce to specific signal assignments that are directly readable from the RTL. Of the 38 requirements, 28 are closed. Table 6 lists the five testbenches and their coverage.

Figure 19 shows the two-node integration scenario from `can_mac_pcs_fce_tb`: a complete FD frame transmitted by DUT 1 and received by DUT 2, with SSP pulses confirming TDC is active during the data phase and resynchronization events visible on DUT 2 ([REQ-009](#)). Figure 20 shows the error escalation scenario: successive ACK errors drive TEC past the 127 threshold, `error_active` deasserts, and `s_suspend_transmission` is inserted after `s_intermission` once the node is error-passive ([REQ-030](#), [REQ-031](#)). Figure 21 and Figure 22 show the bit stuffer's dynamic and fixed mode behavior from `can_mac_bs_tb`, including the mode-boundary promotion case where a pending dynamic stuff bit is promoted to the initial fixed stuff bit rather than suppressed ([REQ-017](#), [REQ-019](#)). Figure 23 shows dual bit rate switching and TDC measurement from `can_pcs_tb`: the PCS replaces nominal segment lengths at the BRS sample point and positions the SSP once the TX-to-RX echo delay is measured ([REQ-025](#), [REQ-026](#)). Figure 24 shows the arbitration loss scenario from `can_mac_pcs_fce_tb`: the losing node clears `is_transmitter` in-place at `s_arbitration` and continues as receiver without a state transition ([REQ-021](#)). Figure 25 shows the active error frame sequence: a forced bit error causes the transmitting node to drive six dominant bits in `s_error_flag`, pulse `fce_o.error` at the transition to `s_error_delimiter`, and count eight recessive bits before resuming ([REQ-022](#), [REQ-023](#)). Figure 26 shows bus-off recovery from `can_fce_tb`: 128 `idle_condition` pulses from the PCS reset TEC and REC to zero and return the node to `s_error_active` ([REQ-031](#)).

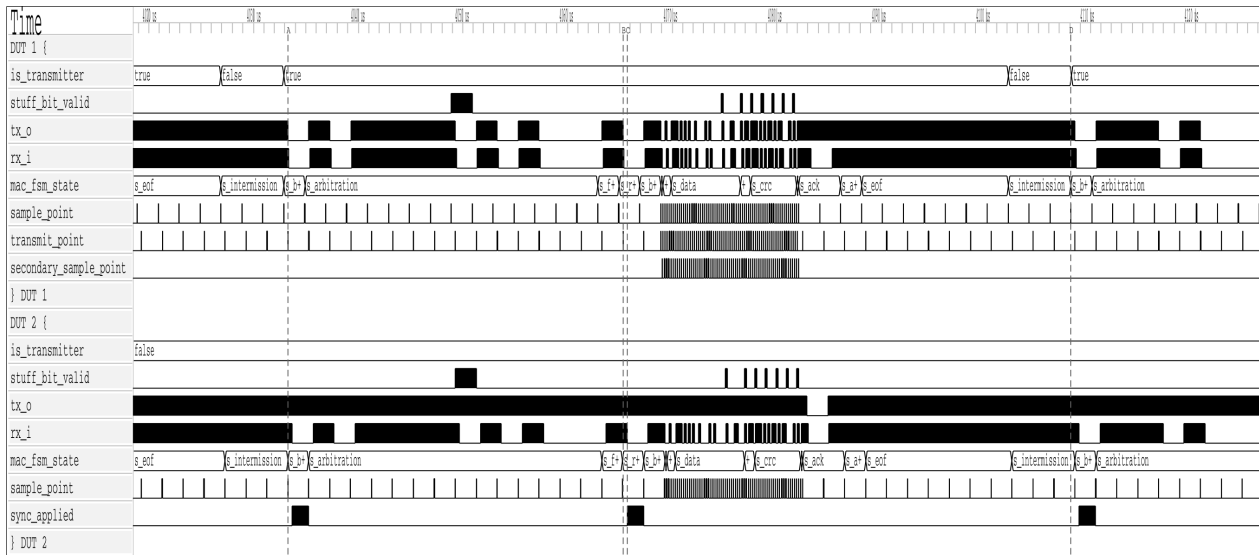


Figure 19: Two-node simulation of a complete FD frame, showing the full FSM field sequence on both transmitter and receiver from `s_arbitration` through bus re-integration.

Ten requirements remain open. Seven are LLC requirements ([REQ-001](#) through [REQ-005](#), [REQ-033](#), [REQ-038](#)) deferred pending implementation of `can_llc`. Two P2 requirements - [REQ-036](#) (MAC data consistency) and [REQ-037](#) (error signaling enable) - are deferred as non-blocking. [REQ-022](#) (error detection, P1) has partial simulation coverage: bit-error detection is exercised via recessive injection in `test_bus_off`,

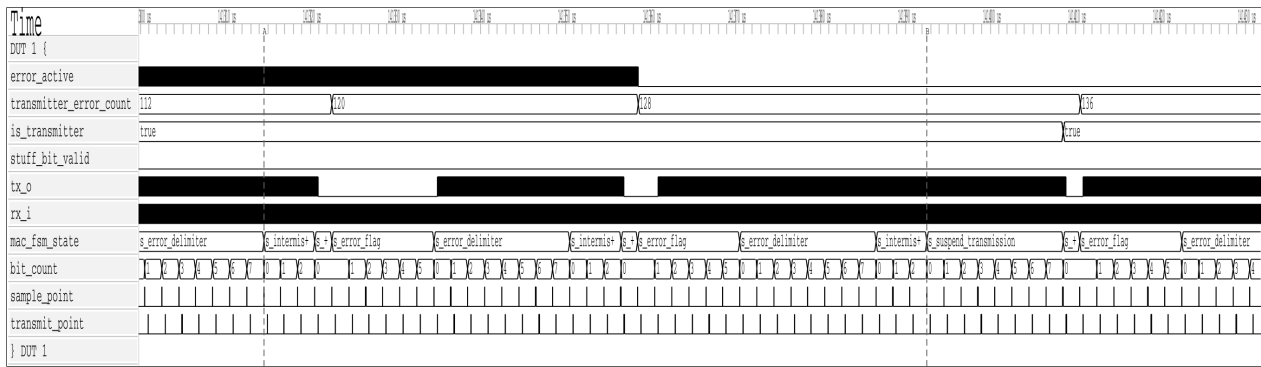


Figure 20: Error escalation waveform showing TEC incrementing across successive ACK errors, `error_active` deasserting at the 127 threshold, and the node transitioning to error passive (REQ-030, REQ-031).

Dynamic bit stuffing in `can_mac_bs_tb`, covering both a dominant-run and a recessive-run case.

Figure 21: Dynamic bit stuffing in `can_mac_bs_tb`, covering both a dominant-run and a recessive-run case.

Fixed bit stuffing mode transition in `can_mac_bs_tb`. A pending dynamic stuff bit at the `fsb_en` rising edge is promoted to the initial FSB rather than suppressed, preventing TX/RX SBC divergence.

Figure 22: Fixed bit stuffing mode transition in `can_mac_bs_tb`. A pending dynamic stuff bit at the `fsb_en` rising edge is promoted to the initial FSB rather than suppressed, preventing TX/RX SBC divergence.

Dual bit rate switching and TDC in `can_pcs_tb`, showing the BRS transition and SSP positioning.

Figure 23: Dual bit rate switching and TDC in `can_pcs_tb`, showing the BRS transition and SSP positioning.

Arbitration loss in `can_mac_pcs_fce_tb`. The losing node clears `is_transmitter` in `s_arbitration` and continues as receiver without a state transition.

Figure 24: Arbitration loss in `can_mac_pcs_fce_tb`. The losing node clears `is_transmitter` in `s_arbitration` and continues as receiver without a state transition.

Active error frame sequence in `can_mac_pcs_fce_tb` following a forced bit error in the data field.

Figure 25: Active error frame sequence in `can_mac_pcs_fce_tb` following a forced bit error in the data field.

Bus-off recovery in `can_fce_tb`, showing the 128 idle-condition pulses required before TEC/REC reset and return to error active.

Figure 26: Bus-off recovery in `can_fce_tb`, showing the 128 idle-condition pulses required before TEC/REC reset and return to error active.

but stuff, form, CRC, and ACK error detection are covered by code inspection only, as each requires a frame-aware stimulus source to inject the error at the correct field boundary (Section 10.2).

8.1 Testbench Results Summary

Table 6: Testbench execution status and requirements coverage.

Testbench	Requirements covered	Status
can_mac_crc_tb	REQ-006	Pass
can_mac_bs_tb	REQ-017, REQ-019	Pass
can_pcs_tb	REQ-025, REQ-026, REQ-027, REQ-028	Pass
can_fce_tb	REQ-029, REQ-030, REQ-031, REQ-035 (sub-claim 2)	Pass
can_mac_pcs_fce_tb	REQ-007, REQ-008, REQ-009, REQ-010, REQ-011, REQ-012, REQ-014, REQ-015, REQ-016, REQ-018, REQ-020, REQ-021, REQ-022 (bit error only), REQ-023, REQ-032, REQ-034	Pass
can_llc_mac_pcs_fce_tb	REQ-035 (sub-claim 1, waveform)	Pass

9 Synthesis

The implemented can_mac_pcs_fce stack was synthesized using Quartus Prime Standard Edition 21.1.1 targeting the Cyclone 10 LP device (10CL016YU256I7G) used in Everllence’s IO-extender board. The synthesis used a standalone project with all record-typed ports flattened to individual std_logic and std_logic_vector signals and all I/O marked as VIRTUAL_PIN, isolating logic resource consumption from I/O buffer overhead. The clock constraint was set to 6 ns (166 MHz), deliberately overconstraining relative to any realistic CAN FD system clock to expose worst-case timing paths. Two constructs in the RTL source required substitution for Quartus compatibility prior to synthesis: conditional signal assignments in sequential processes were replaced with equivalent if/else forms, and the sll operator on std_logic_vector was replaced with concatenation shifts. Both substitutions are functionally equivalent and do not affect resource counts.

9.1 Resource Utilization

The synthesized design uses 4,608 logic elements (30% of the 15,408 available on the target device) and 869 dedicated registers. No memory blocks, embedded multipliers, or PLLs are used. Table 7 shows the per-module resource breakdown.

Table 7: can_mac_pcs_fce resource utilization on Cyclone 10 LP (10CL016YU256I7G).

Module	LEs	Registers	Function
can_mac_fsm	4,109	684	Protocol FSM and RX frame buffer
can_pcs	190	49	Bit timing, TDC, dual bit rate
can_fce	117	31	Fault confinement (TEC/REC)
can_mac_crc	84	53	Three parallel CRC engines
can_mac_ser	84	40	TX serializer
can_mac_bs	31	12	Bit stuffer (dynamic and fixed mode)
Total	4,608	869	

`can_mac_fsm` dominates at 89% of total logic elements. The primary driver is the RX frame buffer: the FSM accumulates received frames into a 70-byte (560-bit) internal byte array, and each buffer register drives combinatorial decode and mux logic. The five remaining modules together consume 506 LEs, confirming that the PCS, FCE, CRC, serializer, and bit stuffer layers add modest overhead relative to the frame buffer cost.

9.2 Timing Results

Table 8 shows the setup and hold slack across all timing corners at the overconstraining 166 MHz clock.

Table 8: Timing results for `can_mac_pcs_fce` at 6 ns (166 MHz) on Cyclone 10 LP.

Corner	Worst setup slack	fmax estimate	Hold slack
Slow 1150 mV 100°C	-1.858 ns	~127 MHz	+0.236 ns
Slow 1150 mV -40°C	-1.550 ns	~152 MHz	+0.021 ns
Fast 1150 mV 100°C	+0.230 ns	>166 MHz	+0.163 ns
Fast 1150 mV -40°C	+0.580 ns	>166 MHz	+0.145 ns

The worst-case fmax is approximately 127 MHz on the slow 100°C corner. The setup failure at 166 MHz is a consequence of the deliberate overconstrain and does not indicate a functional problem at any realistic system clock frequency. A system clock in the 50-80 MHz range typical of Everllence’s IO-extender FPGA designs meets timing with comfortable margin on all corners. Hold slack is positive across all corners. At 30% device utilization on the smallest Cyclone 10 LP variant, the stack fits on the next device step up (10CL025, approximately 19%) or any larger variant in the family.

10 Discussion

The three design-facing verification plan dimensions - `layer`, `side`, and `format_applicability` - shaped the implementation in ways that were not uniformly constructive. `layer` and `format_applicability` motivated sound choices directly: the layered decomposition mapped each requirement to a testable module, and per-field FSM granularity with front-loaded config bytes followed naturally from format applicability analysis. The `side` dimension was a red herring: it made a split TX/RX architecture look well-motivated, but frame structure is the same regardless of which node is driving. Verification plan dimensions are inputs to testbench architecture - which stimulus configurations to exercise - not to RTL decomposition.

The unified FSM paid off most concretely at the arbitration loss boundary: `is_transmitter` clears in place in `s_arbitration` and the shared CRC accumulator and bit stuffer carry over without any handoff. The split-path design required explicit state synchronization at exactly that point, and failed there first (Section 6.2.2). A fix to `can_mac_bs` or `can_mac_crc` propagated to both TX and RX paths automatically. Frame buffer continuity is a further payoff: the `llc_frame` byte array is already being populated from `pcs_i.rx_data` during the arbitration region, so when `is_transmitter` clears on arbitration loss, reception continues into the same buffer with no data to move. A split design would need an explicit transfer mechanism to move partially-accumulated frame data from the TX entity to the RX entity mid-frame.

The layered architecture made white-box requirements tractable. `can_fce_tb` exercised all counter-update rules in isolation. `can_mac_bs_tb` exhaustively covered stuffing mode transitions by driving the stuffer interface directly. `can_mac_pcs_fce_tb` then covered the multi-module scenarios - arbitration loss, error escalation, TDC - that cannot be observed at a single module boundary.

The AI-assisted extraction pipeline introduced a subtle bias that had direct architectural consequences. Classifying each requirement along the side dimension produced a requirements table organized along the TX/RX axis - a faithful representation of the ISO standard, which does frame many obligations in transmitter and receiver terms. But the artifact's structure became an implicit architectural suggestion: a table split along TX/RX lines made a split TX/RX RTL implementation look like the natural realization of the requirements model. The AI did not recommend a split architecture - it simply organized the requirements in a way that made the split appear structurally motivated. The split-path attempt was not the result of bad engineering judgment. It followed a coherent but misleading signal from the requirements artifact. This points to a broader risk in AI-assisted engineering workflows: the structure of an extracted artifact encodes implicit suggestions about downstream decisions, and those suggestions are not labeled as such. Validating the artifact's structure - not just its content - against protocol reality is a step the AI-assisted process does not perform automatically.

The 28/38 requirement closure rate warrants careful interpretation. The 10 open requirements fall into three categories with different implications for production confidence. Seven are LLC requirements deferred pending `can_llc` implementation - a known scope boundary, not a gap in the implemented protocol engine. Two P2 requirements ([REQ-036](#) data consistency, [REQ-037](#) error signaling enable) are non-blocking for the core use case: a node that lacks a configurable error-signaling disable is still a fully correct CAN FD participant.

The synthesis results provide a third lens on the design. The CAN FD stack uses 4,608 logic elements on the Cyclone 10 LP target - 4.0× the 1,146 elements of the existing CAN Classic controller (Section 1.2). This growth is dominated by frame buffer scaling: the RX frame buffer grew from 120 bits (15 bytes, CAN Classic) to 560 bits (70 bytes, CAN FD), a 4.7× increase, driving the combinatorial decode and mux logic that represents the bulk of `can_mac_fsm`'s 4,109 LEs. Excluding the estimated buffer contribution, the protocol logic itself grew approximately 2.9× to deliver dual bit rate switching, TDC, three CRC polynomial variants, fixed bit stuffing with SBC, and full ISO 11898-1 fault confinement - a substantially larger feature set. Normalised for payload capacity, the FD design consumes 72 LEs per payload byte against 143 for CAN Classic, demonstrating better-than-linear scaling with respect to the 8× payload increase. The worst-case f_{max} of approximately 127 MHz on the slow corner provides comfortable timing margin for the 50-80 MHz system clocks typical of Everllence's FPGA designs (Section 9.2). The register-based frame buffer is the straightforward RTL choice but not the efficient one. At 15 bytes (120 bits) in the CAN Classic controller the address decode and read mux overhead is modest - an acceptable side-effect of the implementation approach. At 70 bytes (560 bits) both overhead terms scale proportionally, making them the dominant cost rather than a side-effect: the write-address decoder (the LUT logic that routes a byte index to the correct flip-flop's load enable) and the read mux (the LUT tree that selects the correct byte from 70 parallel register outputs) both grow with buffer depth, and at 70 entries each is large enough to dominate the module's LE count. Mapping the 70-byte buffer to a single block RAM instance would eliminate both the storage flip-flops and the combinatorial decode logic they feed, reducing `can_mac_fsm`'s footprint substantially and leaving only protocol and control logic in the LUT fabric (Section 10.2).

10.1 Objectives Assessment

The four objectives stated in Section 1.5 are assessed against the verification results.

CAN/CAN FD protocol controller in VHDL compliant with ISO 11898-1, supporting CB, CE, FB, and FE frames. The unified `can_mac_fsm` handles all four in-scope frame formats in both transmission and reception, including dual bit rate switching with Transmitter Delay Compensation in the FD data phase

(REQ-025, REQ-026). 28 of 38 requirements are closed. The remaining 10 are LLC-layer requirements deferred pending can_llc implementation, known P2 gaps, or requirements with partial simulation coverage, all documented in Section 10.2.

ISO 11898-1 sub-layer structure enabling independent module verification. The layered decomposition was implemented as designed. Each module has a dedicated testbench and a disjoint requirement set. The five requirements labelled system correctly identified the scenarios requiring multi-module stimulus, confirming that the layer boundaries were drawn at the right points.

Structured requirements with traceability from ISO 11898-1 to testbench results. 38 requirements were derived from ISO 11898-1 normative clauses, each linked to its source section, verification method, testbench file, and assertion label. 28 are closed against passing testbenches or code inspection, establishing a direct traceable path from standard clause to verification artifact. The full plan is reproduced in Section A.

RTL design integrated via Avalon-ST interfaces into Everllence's existing FPGA infrastructure. The RTL source is written in portable VHDL-93 with no vendor primitives. Synthesis on a Cyclone 10 LP target confirmed timing closure at realistic system clock frequencies with 30% device utilization (Section 9). The Avalon-ST host interface is the responsibility of can_llc, which is not yet implemented. Its interface contracts are fully specified in the verification plan.

10.2 Future Work

1. **can_llc implementation.** The LLC sub-layer is the one unimplemented module. Its interface contracts are fully specified in REQ-001 through REQ-005, REQ-033, and REQ-038. Implementing it closes the Avalon-ST host interface and completes the full CAN node.
2. **Hardware integration and bring-up.** The implemented RTL has been verified in simulation only. Integration into Everllence's IO-extender FPGA design and bring-up on a physical CAN FD bus - including interoperability testing against a known-good CAN FD node - would validate timing closure, transceiver compatibility, and bit timing calibration under real bus conditions.
3. **CAN XL support.** CAN XL is explicitly out of scope for this project. The layered architecture and unified FSM are well-suited for extension: CAN XL adds a third bit rate phase and an XL-specific frame format, both of which map naturally onto additional PCS rate parameters and new can_mac_fsm states.
4. **Frame buffer block RAM migration.** The 70-byte RX frame buffer is implemented as a 560-bit flip-flop array, which is the primary driver of can_mac_fsm's 4,109 LEs. A single M9K block RAM instance on the Cyclone 10 LP provides 9 Kbit of dedicated storage with negligible LUT overhead and is sufficient to hold the entire frame. The frame accumulation logic writes sequentially by byte index during reception, and p_stream_to_LLC reads sequentially during the post-EOF transfer - both access patterns map directly to single-port BRAM without any structural change to the surrounding FSM logic. This migration would reduce the LUT footprint to approximately the protocol-logic-only estimate, making the resource cost proportional to CAN FD's actual protocol complexity rather than its frame size.
5. **Error-type-specific simulation coverage (REQ-022).** The current testbench verifies bit-error detection through recessive injection at frame start. Sub-claims 2-5 (stuff, form, CRC, and ACK error detection) are covered by code inspection only. CRC error is the most accessible: flipping any bit in the RX DUT's bus view causes its locally computed CRC to diverge from the transmitted one, with

the mismatch detected when the CRC field is reached. No frame-level timing is required, and this can be added as a targeted extension to `can_mac_pcs_fce_tb`. Stuff, form, and ACK error injection each require a frame-aware stimulus source - stuff error must target a stuff bit, form error must target a fixed-format field, and ACK error must target the ACK slot. Closing these three sub-claims requires either the Everllence-internal CAN FD reference model with targeted error injection capability, or white-box FSM state signals exposed from `can_mac_fsm` to time the injection correctly.

11 Conclusion

This thesis presented the design, implementation, and verification of a CAN/CAN FD protocol controller in VHDL-93, structured around the ISO 11898-1 layered reference model. The implemented design covers the MAC, PCS, and FCE sub-layers as independently testable modules, supports all four in-scope frame formats (CB, CE, FB, FE), implements dual bit rate switching with Transmitter Delay Compensation, and integrates into Everllence’s existing FPGA infrastructure via Avalon-ST interfaces. Of the 38 requirements derived from ISO 11898-1, 28 are closed against passing testbenches or code inspection. Of the remaining 10, seven are deferred pending `can_llc` implementation, two are non-blocking P2 items, and one (REQ-022) has partial simulation coverage with dedicated error-injection testing identified as future work. The design was synthesized on a Cyclone 10 LP FPGA target using 4,608 logic elements (30% of device) with a worst-case fmax of approximately 127 MHz, confirming timing closure at realistic system clock frequencies.

The project yielded three transferable lessons. First, the structure of a requirements model can inadvertently bias RTL architecture: the TX/RX side dimension of the verification plan made a split-path implementation appear well-motivated, but the frame structure of the CAN protocol is the same regardless of which node is driving, and cutting the natural code unit at an artificial seam added coordination complexity without reducing protocol complexity. Verification plan dimensions are inputs to testbench architecture, not to RTL decomposition. Second, the ISO 11898-1 layered architecture is not merely a documentary convenience - it is a practical partitioning of protocol complexity that, when followed in the implementation, enables each sub-layer to be implemented, verified, and debugged independently. The modular design produced here is maintainable over the long product lifecycles that motivate Everllence’s decision to develop the protocol controller in-house. Third, targeted, schema-validated field updates to a structured artifact are safe for an LLM to perform incrementally. Full-file rewrites are not (Section 3.3). The narrow MCP write interface made AI-assisted maintenance of the verification plan safe and practical across all three project phases. The result is an IP core that Everllence owns outright - maintainable, verifiable, and extensible over the multi-decade service commitments that motivated the redesign.

12 References

- [1] M. Jeřábek and P. Piša, “CTU CAN FD — open-source CAN FD IP core.” <https://github.com/Logic-Design-Services/CTU-CAN-FD>, 2019.
- [2] International Organization for Standardization, *Road vehicles — controller area network (CAN) conformance test plan — Part 1: Data link layer and physical signalling*. 2016.
- [3] Robert Bosch GmbH, “M_CAN — CAN FD protocol controller IP module.” <https://www.bosch-semiconductors.com/products/ip-modules/can-ip-modules/m-can/>, 2024.
- [4] AMD (Xilinx), “CAN FD controller — Vivado design suite product guide (PG223).” <https://docs.amd.com/r/en-US/pg223-canfd>, 2024.

- [5] CAST, Inc., “CAN FD protocol controller.” <https://www.cast-inc.com/interfaces/automotive-bus-controllers/can-ctrl>, 2024.
- [6] International Organization for Standardization, *Road vehicles — controller area network (CAN) — Part 1: Data link layer and physical coding sublayer*. 2024.
- [7] Robert Bosch GmbH, “CAN specification version 2.0,” Robert Bosch GmbH, 1991.
- [8] J. Charzinski, “Performance of the error detection mechanisms in CAN,” in *Proceedings of the 1st international CAN conference (iCC 1994)*, Mainz, Germany, 1994.
- [9] F. Hartwich, “CAN with flexible data-rate,” in *Proceedings of the 13th international CAN conference (iCC 2012)*, 2012.
- [10] A. Mutter, “Robustness of a CAN FD bus system — about oscillator tolerance and edge deviations,” in *Proceedings of the 14th international CAN conference (iCC 2013)*, Paris, France, 2013.
- [11] A. Mutter and F. Hartwich, “Advantages of CAN FD error detection mechanisms compared to classical CAN,” in *Proceedings of the 15th international CAN conference (iCC 2015)*, 2015.
- [12] OSVVM Contributors, “OSVVM — open source VHDL verification methodology.” <https://osvvm.org>, 2024.
- [13] GHDL Contributors, “GHDL — open-source VHDL simulator.” <https://ghdl.github.io/ghdl/>, 2024.
- [14] Intel Corporation, “Avalon interface specifications,” Intel Corporation, 683091, 2021. Available: <https://docs.altera.com/r/docs/683091/current>
- [15] J. Bergeron, “Verification planning,” in *Writing testbenches: Functional verification of HDL models*, 2nd ed., Kluwer Academic Publishers, 2003.

A Verification Plan

Both tables are regenerated automatically from `verification_plan/verification_plan.toml` on each PDF build. The ID field is the join key between them. See Section 5.7 for the meaning of each field. The first table lists each requirement with its ISO source clause, priority, and paraphrase. The second table lists the verification metadata: layer, side, format applicability, observability, method, status, traceability label, file, and coverage criteria.

Table 9: ISO 11898-1 extracted requirements.

ID	ISO ref	Priority	Requirement	Notes
REQ-001	6.4.5.2	P1	LLC notification LLC shall notify the user on every successful DF/RF TX or RX.	Timestamping (§6.5.7) is optional per ISO and is out of scope.
REQ-002	6.4.5.5.2, 6.4.5.5.3	P2	LLC transmission request and abort timing 1) A TX request shall be processed no later than the second SOF after the request when no EFs are present. 2) Frames not yet passed to the MAC are aborted immediately. 3) An abort request shall be processed before the second SOF (no EFs present) or third SOF (with EFs present).	Frames already passed to MAC are aborted only on MAC error or arbitration loss.
REQ-003	6.5.2, 6.5.3	P1	Retransmission 1) Frames losing arbitration shall be retransmitted unless aborted by the LLC user. 2) Non-XL nodes shall support unlimited retransmissions.	
REQ-004	6.5.4	P3	Frame acceptance filtering 1) Receivers should use acceptance filtering to decide frame relevance. 2) Filtering should be a single self-contained operation with no state carry-over from previous frames.	
REQ-005	6.5.6	P1	LLC transfer status reporting LLC shall report five transfer status values: Ongoing, Transmitted, Lost_Arbitration, Disturbed, and Aborted, each reflecting the corresponding MAC sub-layer event.	
REQ-006	6.6.4.4	P1	CRC rules 1) CRC_15 for CC frames. CRC_17 for FD frames with payload ≤ 16 B. CRC_21 for FD frames with payload > 16 B. 2) Init vector: all-zero for CRC_15. MSB-one for CRC_17 and CRC_21.	

ID	ISO ref	Priority	Requirement	Notes
REQ-007	6.6.5.2, 6.6.5.3, 6.6.6.2, 6.6.6.3	P1	Error and overload flags 1) Active error and overload flags: 6 consecutive dominant bits each. 2) Passive error flag: 6 consecutive recessive bits (may be overwritten by dominant bits from other nodes). 3) Error and overload delimiters: 8 recessive bits each. 4) After the flag, all nodes monitor the bus and simultaneously begin the remaining 7-bit delimiter on detecting the first recessive bit.	
REQ-008	6.6.7.1, 6.6.7.2, 6.6.7.3, 6.6.7.4	P1	Inter-frame space 1) Intermission shall be exactly 3 recessive bits. 2) No DF or RF may be started during intermission. 3) EFs and OFs shall not be preceded by inter-frame space. 4) Bus idle is recognised at the third recessive intermission bit (error-active nodes/receivers), the eighth recessive suspend bit (error-passive transmitters), or on leaving bus-integration state. 5) An error-passive transmitter shall suspend for 8 bit times after intermission. If another node transmits during the suspend window, the node shall become a receiver.	
REQ-009	6.6.7.5	P1	Bus re-integration 1) Nodes shall start bus re-integration on protocol startup or bus-off recovery. 2) Re-integration completes after 11 consecutive recessive bits at the sample point. 3) For FD-capable nodes, any synchronisation-causing edge shall reset the 11-bit recessive-bit counter.	
REQ-010	6.6.7.3, 6.6.8	P1	Frame initiation 1) SOF shall be a single dominant bit. 2) A node shall send SOF only when the bus is idle. 3) A dominant third intermission bit causes a node with a pending frame (if error-active or previous receiver) to skip SOF and begin arbitration immediately. 4) A dominant bit detected during bus idle shall be interpreted as SOF.	

ID	ISO ref	Priority	Requirement	Notes
REQ-011	6.6.10.1, 6.6.10.4	P2	Remote frame 1) RTR bit shall be dominant for DFs and recessive for RFs. 2) RFs have no data field. DLC indicates the requested data length.	
REQ-012	6.6.10.2, 6.6.11.2	P1	SRR and RRS reserved bits 1) SRR (CE, FE): transmitted recessive. Receivers accept either polarity without form error. 2) RRS (FB, FE): transmitted dominant. Receivers accept either polarity without form error.	
REQ-013	6.6.10.5, 6.6.11.5	P1	CRC bit stream coverage 1) CC: SOF, arbitration, control, and data fields (dynamic stuff bits excluded). 2) FD: SOF, arbitration, control, data, dynamic stuff bits, and stuff count field (fixed stuff bits excluded).	
REQ-014	6.6.10.6, 6.6.11.6, 6.6.14	P1	Frame acknowledgement 1) Receivers shall ACK consistent frames and flag inconsistent frames with an EF. 2) Transmitters shall flag unacknowledged frames with an EF. 3) FD frames tolerate up to a 2-bit dominant ACK overlap to compensate for receiver phase differences. 4) The ACK delimiter shall be one recessive bit. 5) ACK shall not depend on the frame identifier.	
REQ-015	6.6.10.3, 6.6.11.2, 6.6.11.3	P1	CC/FD format differentiation 1) In FD frames the FDF bit is recessive. The overlapping CC bit position (r0 in CBFF, r1 in CEFF) shall be dominant. 2) IDE is in the control field for CB/FB frames, and in the arbitration field for CE/FE frames. 3) The res bit (FB/FE) shall be dominant.	
REQ-016	6.6.11.3	P2	ESI bit transmission 1) Recessive when the LLC ESI flag is set. 2) Dominant when LLC ESI flag is clear and the node is error-active. 3) Recessive when LLC ESI flag is clear and the node is error-passive.	

ID	ISO ref	Priority	Requirement	Notes
REQ-017	6.6.11.5	P1	Stuff bit count (SBC) field 1) SBC: 3-bit Gray-coded count of dynamic stuff bits (0-7) plus parity bit, placed before the CRC field. 2) TX shall transmit its dynamic stuff-bit count (up to the first FSB) as the SBC field. 3) RX shall verify the received SBC against its own count.	
REQ-018	6.6.10.5, 6.6.11.5	P1	CRC delimiter 1) CC frames: exactly one recessive CRC delimiter bit. 2) FD frames: transmitter sends one recessive bit. A second recessive bit is tolerated before the ACK slot.	
REQ-019	6.6.13.1, 6.6.13.2, 6.6.13.3.1	P1	Bit stuffing 1) After 5 consecutive identical bits, insert a complementary dynamic stuff bit. RX discards it. 2) CC: dynamic stuffing spans SOF to FCRC. FD: SOF to end of data field. ACK and EOF are not stuffed. 3) FD CRC field: a fixed stuff bit precedes the SBC field and follows every fourth CRC bit, each the inverse of the preceding bit. No double stuffing. 4) RX discards FSBs and flags a form error if an FSB matches the preceding bit.	
REQ-020	6.6.10.7, 6.6.11.7, 6.6.15.2	P1	End of frame (EOF) 1) EOF shall consist of exactly 7 recessive bits. 2) A receiver shall validate a frame after the sixth EOF bit. 3) A transmitter requires all seven EOF bits to be error-free.	

ID	ISO ref	Priority	Requirement	Notes
REQ-021	6.6.10.2, 6.6.11.2, 6.6.17.4, 6.6.17.5	P1	Bus arbitration 1) Recessive-sent, dominant-monitored: lose arbitration and become a receiver. 2) Arbitration field: CB: base ID + RTR. CE: base ID + SRR + IDE + extended ID + RTR. FB: base ID + RRS. FE: base ID + SRR + IDE + extended ID. 3) CB wins over FB: CBFF sends dominant r0 where FBFF sends recessive FDF. 4) CB/FB wins over CE/FE: CEFF/FEFF sends recessive SRR where CBFF/FBFF sends dominant. 5) (CB, CE only) DF wins over RF with same ID via its dominant RTR bit.	

ID	ISO ref	Priority	Requirement	Notes
REQ-022	6.6.21.2	P1	MAC error detection 1) Bit error: sent/monitored mismatch. Exceptions: recessive non-stuff bit during arbitration, recessive bit in ACK slot, dominant bit while sending a passive error flag. 2) Stuff error: sixth consecutive identical bit in a dynamically-stuffed field. 3) Form error: unexpected fixed-form bit value, or FSB not inverse of preceding bit (FB/FE). Exception: dominant last EOF bit or last delimiter bit is not a form error. 4) CRC error: CRC mismatch. FD also requires SBC match and correct parity. 5) ACK error: transmitter detects no dominant bit in the ACK slot.	Sub-claim 1 (bit error) is simulation-verified via recessive injection in <code>test_bus_off</code> (<code>can_mac_pcs_fce_tb</code>): the bus is forced recessive at frame start, causing a bit error on the first dominant drive. Sub-claim 4 (CRC error) can be triggered by a simple testbench extension: flipping any bit in the RX DUT's bus view causes its locally computed CRC to diverge from the transmitted CRC, with the mismatch detected when the CRC field is reached. No frame-level timing is required. Sub-claims 2 (stuff error), 3 (form error), and 5 (ACK error) each require a frame-aware stimulus source: stuff error must target a stuff bit, form error must target a fixed-format field, and ACK error must target the ACK slot. Closing sub-claims 2, 3, and 5 requires either the Everllence-internal CAN FD reference model with targeted error injection capability, or white-box FSM state signals exposed from <code>can_mac_fsm</code> . All four outstanding sub-claims are covered by code inspection only.
REQ-023	6.6.21.3.1	P1	Error flag initiation and CRC error behaviour 1) On any detected error, start an EF at the next bit and inform the LLC. 2) Data-phase errors: switch to nominal bit rate before starting the EF. 3) CRC error: receiver withholds ACK and sends an EF at the first EOF bit (CC) or 3 bit-times after the CRC delimiter (FD).	

ID	ISO ref	Priority	Requirement	Notes
REQ-024	6.6.21.3.2	P1	Overload frame 1) LLC-requested OF: optional, starts at the first intermission bit, at most 2 per inter-frame space. 2) Reactive OF: triggered by a dominant bit at the first two intermission bits, the last EOF bit (receivers only), or the last error/overload delimiter bit. Starts one bit after the trigger.	
REQ-025	7.3.2, 7.3.3	P1	PCS bit timing configuration 1) Separate nominal and FD data bit rate configurations, each based on a programmable integer prescaler. 2) Sample point at end of Phase_Seg1. Sync_Seg = 1 tq. 3) IPT ≤ 2 tq.	Deployment constraints on the system (not RTL behaviors): prop_seg shall be programmed $\geq t_{\text{node(A)}} + t_{\text{node(B)}} + 2 \times t_{\text{busline}}$ (§7.3.2). SJW shall not exceed min(Phase_Seg1, Phase_Seg2) in either phase (§7.3.2). FD data phase: Phase_Seg2 shall be programmed $\geq$ SJW and $\geq$ maximum IPT (§7.3.3). Oscillator tolerance df shall satisfy $df < SJW / (20 \times t_{\text{bit}})$ and $df < \min(\text{Phase_Seg1}, \text{Phase_Seg2}) / (2 \times (13 \times t_{\text{bit}} - \text{Phase_Seg2}))$ (§7.3.6).
REQ-026	7.3.4, 6.6.21.3.1	P1	Transmitter delay compensation 1) TDC applies in the FD data phase when BRS is recessive. Enable and prescaler (1 or 2) are programmable. 2) SSP position = measured transmitter delay + configured offset. 3) TDC active: bit errors at the SP are ignored. SSP errors are flagged at the following SP.	

ID	ISO ref	Priority	Requirement	Notes
REQ-027	7.3.5.1, 7.3.5.2, 7.3.5.3, 7.3.5.4	P1	PCS synchronisation 1) One synchronisation per bit time, re-enabled after the next recessive SP. 2) Edge qualifies only if the previous SP was recessive and no positive phase error exists while sending dominant. 3) Hard sync: IFS edges (except first intermission bit), bus-integration, and FDF-to-res transition. Restarts bit time with Sync_Seg completed, unconstrained by SJW. 4) All other qualifying edges cause resync, except for the transmitter during the FD data phase. Positive error: Phase_Seg1 $\neq$ SJW. Negative error: Phase_Seg2 $\neq$ SJW. Error $\leq$ SJW: same effect as hard sync.	RTL is stricter than ISO: mac_i.transmitting suppresses all sync unconditionally (line 177), not just resync on positive phase error. Safe since the transmitter is the timing source.
REQ-028	6.6.7.5, 7.4.2.2, 7.4.2.3, 8.1.4.4	P1	PCS bus-off isolation and signalling 1) During bus-off, the PCS shall not drive dominant bits. 2) PCS enters bus-off on a supervisor request and exits on a release request.	
REQ-029	8.1.3.2, Table 14, Table 15	P1	FCE reset and response 1) TEC and REC shall both be reset to zero when the FCE enters its initial state.	
REQ-030	8.1.4.2 rule a), rule b), rule c), rule d), rule e), rule f), rule g), rule h)	P1	Error counter rules 1) REC +1 on any receiver-detected error (except bit errors during error/overload flag). 2) REC +8: first dominant bit after sending an error flag, or bit error while sending an active error/overload flag. 3) REC -1 on successful RX if 1-127. Stays 0 if already 0. Set to 119-127 if above 127. 4) TEC +8 when sending an error flag. Exceptions: error-passive ACK error, pre-arbitration stuff error. 5) TEC +8 on bit error while sending an active error or overload flag. 6) TEC -1 on successful TX (floor 0). 7) After 14 consecutive dominant bits post active error/overload flag (8 post passive error flag): both +8. Each additional 8 dominant bits: both +8.	

ID	ISO ref	Priority	Requirement	Notes
REQ-031	8.1.4.3, 8.1.4.4, 8.1.3.2	P1	Error confinement state transitions 1) Error-passive when either error counter > 127. Error-active restored when both ≤ 127. 2) TEC > 255 causes bus-off. 3) Bus-off recovery requires 128 idle conditions. Both counters reset to zero on recovery. 4) FCE shall notify the LLC on entering bus-off state.	Sub-claim 4: bus-off recovery requires 128 separate idle detections (128 × 11 recessive bits), not 128 consecutive recessive bits.
REQ-032	6.6.11.3	P1	BRS bit rate switching 1) Recessive BRS: switch to FD data rate at the BRS sample point. Dominant BRS: keep nominal rate. 2) Rate switches back to nominal at the first CRC delimiter sample point.	
REQ-033	6.4.3, 6.4.4, Table 5	P1	DLC byte-count mapping DLC 0-8: linear mapping. FD: DLC 9→12, 10→16, 11→20, 12→24, 13→32, 14→48, 15→64 bytes. CC: DLC 9-15 clamped to 8 bytes.	
REQ-034	6.6.16	P1	Byte/Bit transmission order Within the data field byte 0 is sent first. Within each byte bit(7) is sent first.	
REQ-035	8.1.5	P1	Node start-up 1) A node shall complete bus re-integration (11 consecutive recessive bits) before transmitting. 2) A single node receiving no ACK becomes error-passive but never bus-off.	Sub-claim 2 (lone node stays error-passive, never bus-off) is verified by test_rule_c_exception in can_fce_tb via the passive_tx_ack_error_exempt_1 Exception 1 mechanism. Sub-claim 1 (bus re-integration before TX) is verified by waveform inspection of normal_test and test_bus_off in can_llc_mac_pcs_fce_tb: the node is observed completing the 11-consecutive-recessive-bit reintegration phase before any transmission.

ID	ISO ref	Priority	Requirement	Notes
REQ-036	6.6.18	P2	MAC data consistency Frames shall be fully buffered before TX starts, or the LLC shall detect and invalidate frames corrupted mid-transmission.	Verify by code inspection which method the implementation uses and confirm it is applied consistently. If method (a) - confirm the MAC FSM only begins serialisation after the full frame is loaded. If method (b) - confirm the LLC has an error-detection path and can abort or invalidate an in-progress frame.
REQ-037	6.6.21.1, 6.6.21.4	P2	Error signalling enable A configuration parameter shall exist to enable or disable error signalling.	
REQ-038	6.4.4	P3	DLC padding An LLC restricted to fewer bytes shall replace excess bytes with 0xCC padding in both TX and RX directions.	Only applicable if the implementation exposes a configurable maximum-data-byte restriction. If not implemented, waive.

Table 10: ISO 11898-1 verification plan.

ID	Layer	Side	Format	Observability	Method	Status	Label	File	Coverage criteria
REQ-001	LLC	both	CB, CE, FB, FE	black_box	simulation	not_started			
REQ-002	LLC	transmitter	CB, CE, FB, FE	white_box	code_inspection	not_started			
REQ-003	LLC	transmitter	CB, CE, FB, FE	white_box	code_inspection	not_started			
REQ-004	LLC	receiver	CB, CE, FB, FE	white_box	code_inspection	not_started			
REQ-005	LLC	transmitter	CB, CE, FB, FE	black_box	simulation	not_started			
REQ-006	MAC	both	CB, CE, FB, FE	white_box	simulation, coverage	complete	crc_15, crc_17, crc_21	can_mac_crc_tb	Three bins: (1) CB or CE frame (CRC_15), (2) FB or FE frame with data ≤ 16 B (DLC ≤ 10 , CRC_17), (3) FB or FE frame with data > 16 B (DLC ≥ 11 , CRC_21) - must each be hit.
REQ-007	system	both	CB, CE, FB, FE	white_box	code_inspection, wave-form_inspection	complete	p_fsm:~900, test_bus_off	can_mac_pcs_fce_tb	
REQ-008	MAC	both	CB, CE, FB, FE	white_box	code_inspection, wave-form_inspection	complete	p_fsm:~486, test_normal, test_bus_off	can_mac_pcs_fce_tb	
REQ-009	system	both	CB, CE, FB, FE	white_box	code_inspection, wave-form_inspection	complete	p_fsm:~433, test_bus_off	can_mac_pcs_fce_tb	

ID	Layer	Side	Format	Observability	Method	Status	Label	File	Coverage criteria
REQ-010	MAC	both	CB, CE, FB, FE	white_box	code_inspection, wave-form_inspection	complete	p_fsm:~451, test_normal	can_mac_pcs_fce_tb	
REQ-011	MAC	both	CB, CE	black_box	simulation	complete	FTYP Coverage	can_mac_pcs_fce_tb	
REQ-012	MAC	both	CE, FB, FE	white_box	code_inspection, wave-form_inspection	complete	p_fsm:~546, p_fsm:~548, test_normal	can_mac_pcs_fce_tb	
REQ-013	MAC	both	CB, CE, FB, FE	white_box	code_inspection	complete	p_fsm:~299, p_fsm:~216	can_mac_fsm	
REQ-014	system	both	CB, CE, FB, FE	white_box	simulation, wave-form_inspection	complete	test_normal	can_mac_pcs_fce_tb	
REQ-015	MAC	both	CB, CE, FB, FE	white_box	code_inspection, wave-form_inspection	complete	p_fsm:~613, test_normal	can_mac_pcs_fce_tb	
REQ-016	MAC	transmitter	FB, FE	white_box	simulation, wave-form_inspection	complete	test_normal	can_mac_pcs_fce_tb	
REQ-017	MAC	both	FB, FE	black_box	simulation	complete	p_sbc_checker	can_mac_bs_tb	
REQ-018	MAC	transmitter	CB, CE, FB, FE	white_box	code_inspection, wave-form_inspection	complete	p_fsm:~768, p_fsm:~796, test_normal	can_mac_pcs_fce_tb	
REQ-019	MAC	both	CB, CE, FB, FE	white_box	simulation, coverage	complete	Input Coverage, Output Coverage	can_mac_bs_tb	All bins in cov_input and cov_output must be hit.
REQ-020	MAC	both	CB, CE, FB, FE	white_box	simulation	complete	test_normal	can_mac_pcs_fce_tb	
REQ-021	system	transmitter	CB, CE, FB, FE	black_box	simulation	complete	test_lost_arb	can_mac_pcs_fce_tb	

ID	Layer	Side	Format	Observability	Method	Status	Label	File	Coverage criteria
REQ-022	MAC	both	CB, CE, FB, FE	white_box	code_inspection, wave-form_inspection	in_progress	p_fsm:~283, p_fsm:~336, p_fsm:~343, p_fsm:~415, test_bus_off	can_mac_pcs_fce_tb	
REQ-023	MAC	both	CB, CE, FB, FE	white_box	code_inspection, wave-form_inspection	complete	p_fsm:~283, p_fsm:~294, p_fsm:~357, test_bus_off	can_mac_pcs_fce_tb	
REQ-024	MAC	both	CB, CE, FB, FE	white_box	code_inspection	complete	p_fsm:~370	can_mac_fsm	
REQ-025	PCS	both	CB, CE, FB, FE	white_box	simulation	complete	test_normal	can_pcs_tb	
REQ-026	PCS	transmitter	FB, FE	black_box	simulation	complete	p_check_tdc_delay	can_pcs_tb	
REQ-027	PCS	both	CB, CE, FB, FE	white_box	code_inspection, wave-form_inspection	complete	p_can_pcs:~177, p_can_pcs:~228, test_normal	can_pcs_tb	
REQ-028	PCS	both	CB, CE, FB, FE	white_box	simulation, code_inspection	complete	p_can_pcs:~280, p_can_pcs:~379, test_bus_off	can_mac_pcs_fce_tb	
REQ-029	FCE	both	CB, CE, FB, FE	black_box	simulation	complete	test_reset, test_bus_off_recovery	can_fce_tb	Sub-claim 1: TEC/REC reset shall take effect within one clock cycle of the request. Sub-claim 2: verify that Normal_mode_response is asserted after reset completes.

ID	Layer	Side	Format	Observability	Method	Status	Label	File	Coverage criteria
REQ-030	FCE	both	CB, CE, FB, FE	black_box	simulation	complete	test_rule_a, test_rule_b, test_rule_c, test_rule_c_exception, test_rule_d, test_rule_e, test_rule_f_tx, test_rule_f_rx, test_rule_g, test_rule_h	can_fce_tb	Rule h: the '>127 → 119-127' clamp path must be explicitly confirmed in simulation.
REQ-031	FCE	both	CB, CE, FB, FE	black_box	simulation	complete	test_bus_off, test_bus_off_recovery	can_fce_tb	
REQ-032	MAC	both	FB, FE	white_box	code_inspection, wave- form_inspection	complete	p_fsm::~636, p_fsm::~755, test_normal	can_mac_pcs_fce_tb	
REQ-033	LLC	both	CB, CE, FB, FE	black_box	simulation, coverage	not_started			Four non-linear FD DLC values (9, 12, 14, 15 → 12, 24, 48, 64 bytes) plus DLC=8 boundary must each produce a correctly sized frame.
REQ-034	MAC	both	CB, CE, FB, FE	black_box	simulation	complete	test_normal	can_mac_pcs_fce_tb	

ID	Layer	Side	Format	Observability	Method	Status	Label	File	Coverage criteria
REQ-035	system	both	CB, CE, FB, FE	black_box	simulation	complete	test_rule_c_exception; normal_test, test_bus_off (waveform)	can_fce_tb.vhd; can_llc_mac_pcs_fce_tb	Sub-claim 1: confirm no TX before 11 consecutive recessive bits are detected at the SP. Sub-claim 2: single-node scenario with no ACK injection; TEC must reach error-passive but not exceed 255.
REQ-036	MAC	transmitter	CB, CE, FB, FE	white_box	code_inspection	not_started			
REQ-037	MAC	both	CB, CE, FB, FE	white_box	code_inspection	not_started			
REQ-038	LLC	both	CB, CE, FB, FE	black_box	simulation	not_started			